

Projektarbeit

Visualisierung kinematischer Kennwerte

An der Fachhochschule Dortmund
im Fachbereich Informatik
Studiengang BA Informatik
6. Fachsemester

von

Philipp Jan Mikos

geb. am 18.02.1993

Matr.-Nr. 7215204

Betreuer: Prof. Dr. Christof Röhrig

Dortmund, 16. Mai 2025

Inhaltsverzeichnis

1	Einleitung	1
1.1	Motivation	1
1.2	Ziel	2
1.3	Aufbau der Arbeit	3
2	Theoretischer Hintergrund	3
2.1	Darstellung der Position und Orientierung eines Roboters	3
2.2	Euler-Winkel und Roll-Pitch-Yaw-Winkel	5
2.3	Rotationsmatrizen	6
2.4	Quaternionen und Unit-Quaternionen	7
2.5	Differentialantrieb	8
2.6	JupyterLab	9
3	Anforderungen	9
4	Aufbau des Frameworks	11
4.1	System- und Softwareumgebung	11
4.2	Framework-Library (Implementierungsdetails)	15
4.3	Beispiel-Notebooks	16
4.3.1	robot.ipynb	16
4.3.2	rotations.ipynb	19
4.3.3	quader.ipynb	20
5	Entwicklungsprozess	24
5.1	Wahl der Rendering-API	24
5.1.1	OpenGL	24
5.1.2	WebGL	25
5.1.3	Babylon.js	25
5.1.4	Three.js	26
5.2	Three.js-Prototyp	26
5.3	Pythreejs-Prototyp	29
5.4	Numpy als Mathematik-Library	31

5.5	Animationslogik	32
5.6	Environment-Klasse	33
6	Evaluation	34
7	Ausblick auf die Bachelorarbeit	35
8	Anhang	1
8.1	Code-Dokumentation der Python-Bibliothek util	1

Abbildungsverzeichnis

1	Ein Radgetriebener Roboter mit einem Körpereigenem Koordinaten- system in der X-Y-Ebene	4
2	Eulerrotation eines Würfels um X, Y und Z in dieser Reihenfolge	5
3	Verteilungsdiagramm des Frameworks	13
4	Roboter mit Differentialantrieb (robot.ipynb während der Ausführung)	16
5	Roboter mit Differentialantrieb (Bild aus der Übungsaufgabe)	18
6	Würfel mit Schiebereglern für die Transformation (rotation.ipynb während der Ausführung)	19
7	Quader aus der Klausuraufgabe	21
8	Quader aus der Klausuraufgabe (robot.ipynb während der Ausführung)	22
9	Verteilungsdiagramm des Prototyps	27
10	Prototyp mit Three.js	28
11	Notebook-Prototyp mit Pythreejs	30

1 Einleitung

Die Grundlagen der *Kinematik*¹ stellen einen wesentlichen Bestandteil der Robotik dar und sind sowohl für mobile als auch für stationäre Roboter von zentraler Bedeutung. Während es bei stationären Robotern häufig darum geht, die Position des *Endeffektors*² präzise zu berechnen, konzentriert sich die Kinematik bei mobilen Robotern vor allem darauf, deren Position zuverlässig bestimmen zu können. Ein fundiertes Verständnis der erforderlichen kinematischen Modelle ist unerlässlich, um diese Größen korrekt zu berechnen (Siegwart u. a. (2011, S. 57)).

Im Rahmen dieser Projektarbeit wird ein *Framework*³ entwickelt, das die Visualisierung kinematischer Kennwerte ermöglicht. Das Ziel dieses Frameworks ist es, unterstützend in der Lehre eingesetzt zu werden, um das Verständnis der Kinematik zu fördern und die praktische Anwendung dieser Konzepte zu erleichtern.

1.1 Motivation

Mein Interesse an Computergrafik entstand vor einigen Jahren durch erste Projekte im Bereich der Spieleentwicklung. Die Möglichkeit, visuelle Inhalte programmatisch zu steuern und darzustellen, hat mich schnell fasziniert und mich tiefer in das Thema hineingezogen. Im Studium kam dann das Themenfeld der Robotik hinzu, das mich vor allem durch die Verbindung von mathematischen Modellen und realer Bewegung begeistert hat. Im Robotik-Modul fragte ich den betreuenden Professor, ob ich meine Projektarbeit in seinem Fachbereich schreiben könne. Er stimmte zu und gab mir die Aufgabe, ein Visualisierungsframework zu entwickeln, mit dem Roboter simuliert werden können. Das Thema war für mich besonders spannend, weil es meine beiden Interessensgebiete Computergrafik und Robotik auf sinnvoll zusammenbringt. Daraus entstand die Motivation, ein praktisches Werkzeug zu entwickeln das hilft kinematische Modelle anschaulich zu vermitteln.

¹Die Kinematik beschreibt die Bewegung von Körpern (z.B. Robotern). Sie beschäftigt sich mit Größen wie Position, Rotation, Geschwindigkeit und Beschleunigung.

²In der Robotik bezeichnet der Endeffektor das letzte Glied eines Roboters, das direkt mit der Umgebung interagiert. Häufig handelt es sich dabei um Werkzeuge oder Greifer.

³In diesem Kontext ist ein Framework eine Sammlung vorgefertigter Bausteine, die die Entwicklung von Software erleichtert. Es gibt den grundsätzlichen Aufbau vor und erlaubt es, eigene Funktionen einzufügen.

Obwohl kinematische Modelle in vielen Bereichen der Robotik eine zentrale Rolle spielen, fällt es Studierenden häufig schwer diese abstrakten mathematischen Konzepte vollständig zu durchdringen. Besonders Rotationen im dreidimensionalen Raum, die durch Eulerwinkel, Roll-Pitch-Yaw-Darstellungen, Rotationsmatrizen oder Quaternionen beschrieben werden, wirken in der Theorie oft wenig anschaulich und sind ohne visuelle Unterstützung schwer intuitiv erfassbar (Gutiérrez u. a. (2016, S. 94–95)). In der Hochschullehre entsteht dadurch eine Lücke zwischen theoretischem Wissen und praktischem Verständnis. Lehrende stehen vor der Herausforderung, komplexe Zusammenhänge nicht nur formal korrekt, sondern auch nachvollziehbar und greifbar zu vermitteln. Gleichzeitig benötigen Studierende Gelegenheiten, das Gelernte interaktiv zu erproben, um eine tiefere, anwendungsorientierte Intuition für die Kinematik zu entwickeln.

Moderne Lehrmethoden, die auf Visualisierung und Interaktion setzen, können hier entscheidend zur Verbesserung des Lernerfolgs beitragen. Insbesondere browserbasierte und plattformunabhängige Lösungen, wie sie in JupyterLab realisierbar sind, eröffnen neue Perspektiven für eine zeitgemäße und praxisnahe Vermittlung technischer Inhalte (Gutiérrez u. a. (2016, S. 239–240, 407)).

1.2 Ziel

Vor diesem Hintergrund zielt das Projekt darauf ab, ein interaktives Visualisierungs-Framework zu entwickeln, das die Lehre im Bereich der Robotik an der Fachhochschule Dortmund unterstützt und bereichert. Das Framework wird speziell für den Einsatz innerhalb der JupyterLab-Umgebung konzipiert und soll dazu dienen, kinematische Zusammenhänge auf intuitive und anschauliche Weise darzustellen.

Ein zentrales Anliegen ist es, Lehrpersonen und Aufgabenstellern ein Werkzeug an die Hand zu geben, mit dem sie durch einfache Python-Befehle individualisierbare Aufgabenstellungen und Übungsszenarien gestalten können. Diese sollen es Studierenden ermöglichen, eigenständig mit Parametern zu experimentieren, Bewegungsabläufe zu analysieren und dadurch ein tieferes Verständnis für die Wirkungsweise kinematischer Modelle zu entwickeln.

Der Fokus dieser Arbeit liegt auf der Rotation von Körpern im Raum. Dabei werden verschiedene Rotationsmodelle wie Eulerwinkel, Roll-Pitch-Yaw-Winkel, Rotationsma-

trizen und Quaternionen behandelt. Durch deren visuelle Umsetzung sollen sowohl Unterschiede als auch Gemeinsamkeiten dieser Darstellungsformen deutlich gemacht und ihre Bedeutung im räumlichen Kontext anschaulich vermittelt werden. Ziel ist es, den Lernprozess im Bereich der Kinematik durch moderne, interaktive Lehrmittel zu unterstützen und einen Beitrag zu einer zeitgemäßen, verständnisorientierten Ausbildung in der Robotik zu leisten.

1.3 Aufbau der Arbeit

Zu Beginn wird in Kapitel 2 auf den theoretischen Hintergrund eingegangen, der notwendig ist, um dieser Arbeit folgen zu können. Dazu zählen die Darstellung der Position und Rotation eines Roboters, Euler- sowie Roll-Pitch-Yaw-Winkel, Rotationsmatrizen, Quaternionen, der Differentialantrieb und die Entwicklungsumgebung JupyterLab. Anschließend werden in Kapitel 3 die Anforderungen spezifiziert, welche die im Rahmen dieser Projektarbeit entwickelte Software erfüllen soll. In Kapitel 4 folgt eine Erläuterung des Aufbaus der Software, bevor in Kapitel 5 der gesamte Entwicklungsprozess beschrieben wird. Daraufhin wird die Software im Hinblick auf die definierten Anforderungen evaluiert. Abschließend wird aufgezeigt, wie dieses Projekt im Rahmen einer Bachelorarbeit weitergeführt werden könnte.

2 Theoretischer Hintergrund

In diesem Kapitel werden die theoretischen Grundlagen vorgestellt, die für das Verständnis und die Umsetzung der in dieser Arbeit behandelten Konzepte notwendig sind. Dazu gehören Methoden zur Beschreibung der Position und Orientierung eines Roboters sowie verschiedene mathematische Modelle für Rotationen. Zudem wird der Differentialantrieb als typisches Fortbewegungsprinzip mobiler Roboter erläutert. Ergänzend wird das Softwarewerkzeug JupyterLab eingeführt, das in dieser Arbeit zur interaktiven Entwicklung und Visualisierung verwendet wird.

2.1 Darstellung der Position und Orientierung eines Roboters

Ein Roboter wird in der Regel als ein unverformbarer fester Körper (Rigid Body) betrachtet, der sowohl eine bestimmte Position als auch eine Orientierung im Raum

besitzt. Die Position beschreibt den Abstand und die Verschiebung zwischen dem globalen Weltkoordinatensystem und dem körpereigenen Koordinatensystem des Roboters. Die Orientierung hingegen gibt den Unterschied in der Winkelausrichtung zwischen dem Weltkoordinatensystem und dem lokalen Koordinatensystem des Roboters an. Die Kombination aus Position und Orientierung eines Körpers wird als Pose bezeichnet. Die Transformation von einer Pose in eine andere kann intuitiv als Bewegung interpretiert werden. Diese Transformation setzt sich aus einer Translation (Verschiebung) und einer Rotation (Drehung) zusammen (Corke (2023, S. 24–27)). Speziell bei radgetriebenen Robotern wird die Pose auf einer Ebene beschrieben. In diesem Fall besteht die Pose aus den X- und Y-Koordinaten im globalen Koordinatensystem sowie dem Winkel zwischen dem globalen und dem körpereigenen Koordinatensystem. Damit lässt sich die Pose als ein Vektor aus diesen drei Elementen darstellen (Siegwart u. a. (2011, S. 59)).

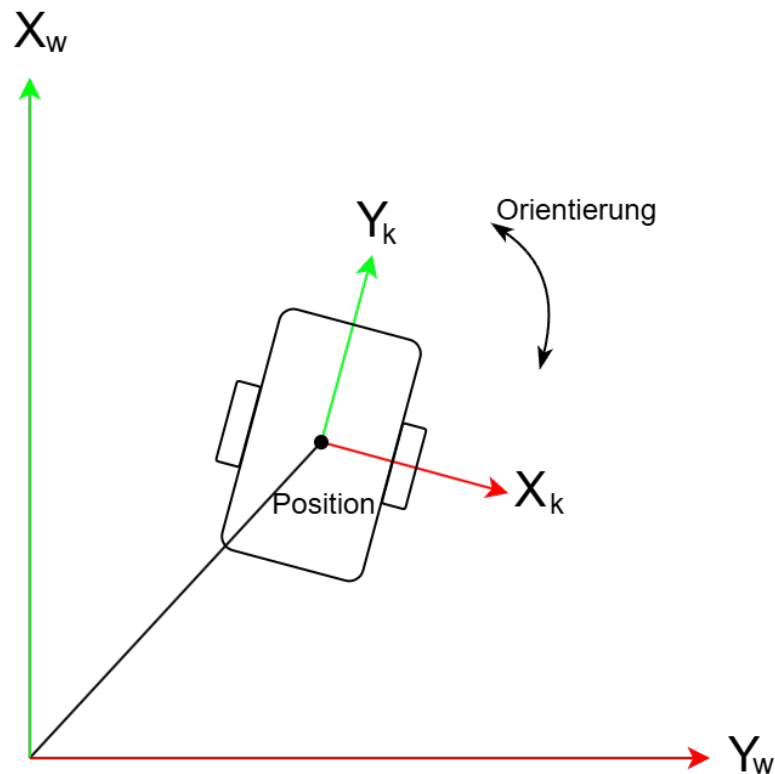


Abbildung 1: Ein Radgetriebener Roboter mit einem Körpereigenem Koordinatensystem in der X-Y-Ebene

2.2 Euler-Winkel und Roll-Pitch-Yaw-Winkel

Euler's Rotationstheorem besagt, dass zwei beliebige unabhängige, orthonormale Koordinatensysteme durch eine Abfolge von maximal drei Rotationen um Koordinatenachsen miteinander in Beziehung gesetzt werden können, wobei keine zwei aufeinanderfolgenden Rotationen um die gleiche Achse erfolgen dürfen (Kuipers (1999, S. 136)). Das bedeutet, dass jede Rotation zwischen zwei Koordinatensystemen im dreidimensionalen Raum durch eine Sequenz von drei Rotationswinkeln beschrieben werden kann, wobei jeder Winkel einer bestimmten Achse zugeordnet ist. Die Rotationen werden nacheinander ausgeführt: Zunächst erfolgt eine Rotation des Weltkoordinatensystems um eine Achse, wodurch ein neues Koordinatensystem entsteht. Anschließend wird dieses neue System um eine seiner eigenen Achsen rotiert, gefolgt von einer dritten Rotation um eine Achse des wiederum neu entstandenen Systems. Dieser Vorgang wird in Abbildung 2 illustriert.

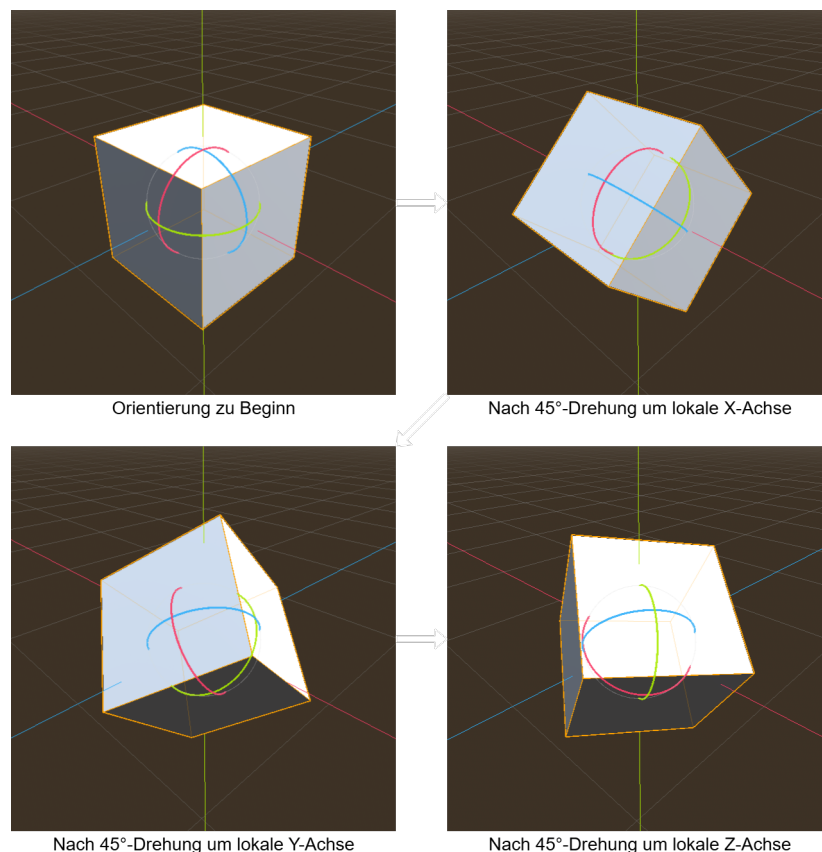


Abbildung 2: Eulerrotation eines Würfels um X, Y und Z in dieser Reihenfolge

In der Abbildung ist ein Würfel zu sehen, der durch drei aufeinanderfolgende Ro-

tationen um sein jeweils aktuelles, lokales Koordinatensystem gedreht wird. Zunächst erfolgt eine Rotation um 45° um die X-Achse. Anschließend wird der Würfel um die daraus resultierende Y-Achse rotiert, gefolgt von einer letzten Drehung um die daraus entstehende Z-Achse. Im Inneren des Würfels befindet sich ein sogenanntes Gimbal (kardanische Aufhängung), dessen Ringe anschaulich verdeutlichen, in welcher Richtung die jeweilige Rotationsbewegung erfolgt.

Insgesamt gibt es zwölf eindeutige Rotationsabfolgen. Sechs davon beinhalten Wiederholungen derselben Achse (aber nicht unmittelbar hintereinander), beispielsweise XXY , XZX , YXY , YZY , ZXZ oder ZYZ . Die anderen sechs Rotationsabfolgen verwenden jeweils alle drei Achsen einmal, beispielsweise XYZ , XZY , YZX , YXZ , ZXY oder ZYX .

Der Begriff Euler-Winkel wird häufig verwendet, um jede Art von dreifacher Rotationsdarstellung zu bezeichnen. Diese Bezeichnung ist jedoch nicht präzise, da je nach Reihenfolge der Rotationen verschiedene Darstellungen entstehen. Deshalb muss die genaue Rotationsabfolge angegeben werden, auch wenn innerhalb bestimmter technischer Fachgebiete wie der Robotik meist die Konvention ZYZ herrscht.

Eine besondere Rolle spielen die sogenannten Roll-Pitch-Yaw-Winkel, bei denen es zwei gebräuchliche Rotationssequenzen gibt: ZYX und XYZ . In der mobilen Robotik wird typischerweise die ZYX -Sequenz verwendet, während bei stationären Roboter manipulatoren oft die XYZ -Sequenz Anwendung findet (Corke (2023, S. 48–51)).

2.3 Rotationsmatrizen

Eine Rotationsmatrix ist eine spezielle Art von Matrix, die ein Koordinatensystem um einen Winkel θ dreht und es dadurch in ein neues Koordinatensystem transformiert. Die Spalten einer Rotationsmatrix können dabei als die transformierten Einheitsvektoren des ursprünglichen Koordinatensystems interpretiert werden.

Rotationsmatrizen sind stets orthonormal, das bedeutet, ihre Spalten (bzw. Zeilen) stehen senkrecht aufeinander und haben die Länge 1. Daraus folgt die wichtige Eigenschaft:

$$R^{-1} = R^T$$

wobei R^{-1} die Inverse und R^T die Transponierte der Matrix R ist. Ein weiteres cha-

rakteristisches Merkmal von Rotationsmatrizen ist, dass ihre Determinante stets den Wert 1 hat.

Desweiteren sind sie Elemente der sogenannten speziellen orthogonalen Gruppe der Dimension 2, die als $SO(2)$ bezeichnet wird. Dies wird auch geschrieben als:

$$R \in SO(2) \subset \mathbb{R}^{2 \times 2}$$

Die spezielle orthogonale Gruppe ist eine Gruppe bezüglich der Matrixmultiplikation. Das bedeutet, dass das Produkt zweier Rotationsmatrizen wieder eine Rotationsmatrix ist und somit ebenfalls zur Gruppe gehört. Ebenso ist die Inverse jeder Rotationsmatrix ebenfalls Teil der Gruppe (Corke (2023, S. 33)).

2.4 Quaternionen und Unit-Quaternionen

Quaternionen wurden im Jahr 1843 von William Rowan Hamilton entdeckt. Sie erweitern die komplexen Zahlen und bestehen aus einem Skalar- und Vektoranteil:

$$q = s + v_x i + v_y j + v_z k$$

wobei i , j und k besondere Einheiten mit $i^2 = j^2 = k^2 = ijk = -1$ sind. Der Skalaranteil ist eine einzelne reelle Zahl, während der Vektoranteil aus drei Komponenten besteht und einem dreidimensionalen Vektor entspricht. Quaternionen bilden einen Vektorraum: Man kann sie addieren, skalieren, multiplizieren (Hamilton-Produkt) und konjugieren. Die Multiplikation ist nicht kommutativ. Der Betrag eines Quaternions ist:

$$||q|| = \sqrt{s^2 + v_x^2 + v_y^2 + v_z^2}$$

Unit-Quaternionen sind Quaternionen mit Betrag 1. Sie werden häufig verwendet, um Rotationen im Raum darzustellen. Eine Rotation um eine Achse $\hat{v}$ mit Winkel θ wird als

$$q = \cos\left(\frac{\theta}{2}\right) + \hat{v} \sin\left(\frac{\theta}{2}\right)$$

bezeichnet. Quaternionen bieten bei der Darstellung von Rotationen mehrere Vorteile. Sie ermöglichen eine recheneffiziente Kombination von Rotationen und vermeiden dabei Probleme, wie den sogenannten Gimbal Lock, der bei der Verwendung von Euler-

Winkeln auftreten kann. Dabei handelt es sich um den Verlust eines Freiheitsgrads, wenn zwei Rotationsachsen durch bestimmte Winkelkonstellationen aufeinander fallen und dadurch nicht mehr unabhängig voneinander steuerbar sind. Zudem sind Quaternionen kompakter als Rotationsmatrizen und benötigen weniger Speicherplatz und Rechenleistung, was sie besonders attraktiv für Anwendungen in Bereichen wie Robotik, Computer Vision, Computergrafik und Navigation macht (Corke (2023, S. 59–61)). Neben der Beschreibung von Rotationen ist für mobile Roboter auch die Modellierung ihrer Fortbewegung von zentraler Bedeutung. Ein häufig genutztes Modell ist dabei der Differentialantrieb.

2.5 Differentialantrieb

Ein mobiler Roboter mit Differentialantrieb besitzt zwei angetriebene Räder, jeweils eines auf jeder Seite. Größere Roboter können zusätzlich über ein oder mehrere Räder auf jeder Seite verfügen, die entweder passiv sind oder mechanisch mit den aktiven Rädern gekoppelt werden, sodass sie sich einen Motor teilen. Die Räder sind an einer festen Position am Roboter befestigt. Die Steuerung des Roboters erfolgt durch die unterschiedliche Antriebsgeschwindigkeit der Räder. Drehen sich beide angetriebenen Räder mit gleicher Geschwindigkeit, bewegt sich der Roboter geradeaus. Rotieren die Räder hingegen mit unterschiedlichen Geschwindigkeiten, führt dies zu einer Drehbewegung des Roboters. Bei Robotern mit Differentialantrieb wird üblicherweise davon ausgegangen, dass es sich bei den verbauten Rädern um Standardräder handelt. Das bedeutet, dass sich die Räder und somit der gesamte Roboter ausschließlich in x -Richtung der körpereigenen Koordinatensysteme der Räder bewegen können, während Bewegungen oder ein Rutschen in y -Richtung nicht möglich sind. Zudem wird angenommen, dass jedes Rad nur einen einzigen Kontaktpunkt mit der Ebene besitzt (Corke (2023, S. 140–141)) (Hertzberg u. a. (2012, S. 107)). Um die beschriebenen kinematischen Modelle und Bewegungsprinzipien interaktiv erfahrbar zu machen, werden geeignete Werkzeuge zur Visualisierung und Simulation benötigt. Eine zentrale Rolle spielt dabei die im Rahmen dieses Projekts verwendete Entwicklungsumgebung JupyterLab, die im folgenden Abschnitt näher erläutert wird.

2.6 JupyterLab

JupyterLab ist eine leistungsstarke und erweiterbare Umgebung zur Erstellung und Bearbeitung von sogenannten computational notebooks. Es ist Teil des größeren Project Jupyter, das sich zum Ziel gesetzt hat, Werkzeuge und Standards für das interaktive Arbeiten mit Rechen-Notizbüchern zu entwickeln.

Ein computational notebook ist ein Dokument, das Computer-Code, Freitext-Beschreibungen, Daten, grafische Darstellungen (z.B. Diagramme, 3D-Modelle) sowie interaktive Bedienelemente miteinander kombiniert. In Verbindung mit einem Editor wie JupyterLab ermöglichen diese Notebooks einen schnellen und interaktiven Entwicklungsprozess, der sich besonders gut für das Prototyping von Code, die Datenexploration, die Visualisierung und das Teilen von Ideen eignet.

JupyterLab ist eng verwandt mit anderen Anwendungen, die dem Project Jupyter entspringen. Dazu zählen zum Beispiel Jupyter Notebook und Jupyter Desktop. Im Vergleich zu Jupyter Notebook bietet JupyterLab jedoch eine fortschrittlichere, funktionsreichere und individuell anpassbare Benutzererfahrung (Project-Jupyter (2024)).

3 Anforderungen

Das zu entwickelnde Framework soll eine Python-Schnittstelle enthalten, da Python eine moderne, weit verbreitete und vor allem gut lesbare Programmiersprache ist, die vielen Studierenden vertraut ist. Die gute Lesbarkeit und klare Syntax sind für ein interaktives Framework wie dieses von Vorteil (Ernesti und Kaiser (2017, S. 35–36)). Das Framework soll für die Nutzung innerhalb der JupyterLab-Umgebung konzipiert sein, da dadurch eine gute Portabilität gewährleistet wird. Außerdem ist JupyterLab bereits in den Robotik-Übungen an der Fachhochschule Dortmund im Einsatz.

Es wird vorausgesetzt, dass Transformationen von Körpern im dreidimensionalen Raum visualisiert werden können. Diese Visualisierungen sollen bevorzugt direkt innerhalb von JupyterLab erfolgen. Sollte das nicht möglich sein, wäre es möglich die visuelle Darstellung in einem separaten Browserfenster vorzunehmen. Für die Visualisierung soll keine lokale Programminstallation erforderlich sein, da dies die Portabilität einschränken würde. Darüber hinaus soll das Framework so gestaltet sein, dass es die komfortable Erstellung und Anwendung von Übungsaufgaben unterstützt und verein-

facht. Zusätzlich wird gefordert, dass das Framework mit *SymPy*⁴ kompatibel ist, da das Framework in bereits vielen schon existierende Notebooks mit Übungsaufgaben integriert werden könnte.

Das Framework soll grundlegende Konzepte der räumlichen Transformation abdecken. Dazu zählen insbesondere Rotationen, Eulerwinkel, Roll-Pitch-Yaw-Winkel sowie Quaternionen zur Darstellung und Analyse von Orientierungen im dreidimensionalen Raum. Darüber hinaus ist vorgesehen, auch einfache Modelle radgetriebener Roboter zu integrieren, um kinematische Zusammenhänge und Bewegungsabläufe anschaulich darzustellen.

Langfristig ist geplant, das Framework um Funktionalitäten zur Modellierung stationärer Roboter, insbesondere Knickarmroboter, zu erweitern. In diesem Zusammenhang sollen auch die Vorwärts- und Rückwärtstransformationen behandelt werden. Aufgrund des Umfangs und der Komplexität dieser Themen wird dieser Teil voraussichtlich im Rahmen einer späteren Bachelorarbeit realisiert und vertieft.

Zwar existieren bereits etablierte Werkzeuge wie beispielsweise Peter Corke's Robotics Toolbox for Python, welche eine Vielzahl an Funktionen für die Modellierung und Analyse robotischer Systeme bieten. Allerdings ist die Nutzung dieser Toolbox im Kontext von JupyterLab nur eingeschränkt möglich. Die dort verwendeten Visualisierungen basieren auf einem separaten *Renderer*⁵ namens Swift, welcher ein eigenes Browserfenster zur Darstellung benötigt und somit nicht direkt in JupyterLab eingebettet ist (Corke (2023, S. 77)). Darüber hinaus ist die Toolbox sehr umfangreich und adressiert viele Konzepte, die für die Anforderungen und Inhalte dieses Projekts nicht von unmittelbarer Relevanz sind.

Auch Tools wie CoppeliaSim bieten umfangreiche Simulationsmöglichkeiten für Robotersysteme, inklusive physikbasierter Modellierung, Sensorik und komplexer Steuerungsarchitekturen. CoppeliaSim ist ein eigenständiger 3D-Robotersimulator mit einer integrierten Skriptumgebung. Es kann sowohl als eigenständige Anwendung genutzt

⁴Sympy ist eine freie Python-Bibliothek für symbolische Mathematik, die es ermöglicht, mathematische Ausdrücke zu vereinfachen oder Gleichungen symbolisch zu lösen.

⁵Ein Renderer ist ein Software- oder Hardware-System, das zur Erzeugung von Bildern oder Szenen aus einem 3D-Modell verantwortlich ist. Es berechnet, wie Licht mit den Objekten interagiert und wie diese auf der 2D-Oberfläche eines Displays dargestellt werden. In der Computergrafik werden verschiedene Rendering-Techniken verwendet, um realistische oder stilisierte Darstellungen zu erzeugen.

als auch in eine Hauptanwendung eingebettet werden. Die Software bietet eine umfangreiche API und eine kompakte Struktur, die eine einfache Integration ermöglicht. Für die Nutzung ist jedoch eine lokale Installation erforderlich, da Coppeliasim als Desktop-Anwendung konzipiert ist. Die Installation umfasst die Einrichtung der Simulationsumgebung sowie der erforderlichen Komponenten für die Skriptverarbeitung und Visualisierung. Im Kontext dieses Projekts, das auf eine browserbasierte und plattformunabhängige Lösung abzielt, stellt die Notwendigkeit einer lokalen Installation eine Einschränkung dar. Zudem ist Coppeliasim auf die Simulation vollständiger Roboterumgebungen ausgerichtet, was über die Anforderungen dieses Projekts hinausgeht (CoppeliaRobotics (2025)).

Eine maßgeschneiderte Lösung, die gezielt auf die Bedürfnisse und Themen des Lehrkonzepts an der Fachhochschule Dortmund zugeschnitten ist, bietet daher klare Vorteile. Sie erleichtert die Erstellung spezifischer Übungsaufgaben durch den Aufgabensteller, fördert die Anschaulichkeit durch eingebettete Visualisierungen für Lernende und trägt insgesamt zur Effizienz und Qualität der Lehre im Bereich der robotischen Transformationen bei.

4 Aufbau des Frameworks

Im folgenden Abschnitt werden die verschiedenen Technologien, die für die Erstellung des Frameworks verwendet wurden, aufgeführt und ihre Bedeutung für das Projekt erläutert. Danach folgt eine detaillierte Beschreibung des technischen Aufbaus, einschließlich der Framework-Library und ihrer wichtigsten Implementierungsdetails. Abschließend werden die Beispiel-Notebooks sowie deren Funktionen und Einsatzmöglichkeiten erklärt.

4.1 System- und Softwareumgebung

Das Framework wurde für die Nutzung in der JupyterLab-Umgebung entwickelt, um eine plattformunabhängige und benutzerfreundliche Arbeitsumgebung zu bieten. Durch den Einsatz von Tools wie Poetry zur Verwaltung von Abhängigkeiten und Bibliotheken wie ipywidgets und pythreejs zur interaktiven 3D-Visualisierung wird eine stabile und effiziente Nutzung ermöglicht.

Kategorie	Technologie	Beschreibung
Programmiersprache	Python	Hochsprachige, interpretierte Programmiersprache mit Fokus auf Lesbarkeit und Einfachheit. Ideal für wissenschaftliches Rechnen, Datenanalyse und Prototyping.
Laufzeitumgebung	JupyterLab	Web-basierte Entwicklungsumgebung für interaktive Notebooks.
Paketmanager	Poetry	Verwaltung von Abhängigkeiten und virtuellen Umgebungen.
Mathe-Bibliothek	Numpy	Bibliothek für numerische Berechnungen und lineare Algebra.
Mathe-Bibliothek	Sympy	Bibliothek für symbolische Berechnungen.
Mathe-Bibliothek	Scipy	Erweiterte Funktionen für wissenschaftliches Rechnen z.B Umwandlung von Rotationsdarstellungen.
3D-Rendering	pythreejs	Python-Wrapper für three.js zur Darstellung von 3D-Geometrien in Notebooks.
Interaktivität	ipywidgets	Erlaubt die Erstellung interaktiver GUI-Elemente wie Schieberegler, Dropdowns und Buttons in Jupyter-Notebooks.

Tabelle 1: Überblick über die im Projekt verwendeten Softwaretechnologien

Tabelle 1 gibt einen Überblick über die zentralen Softwaretechnologien, die für die Implementierung des Frameworks verwendet wurden. Diese Komponenten unterstützen verschiedene Aspekte wie numerische Berechnungen, interaktive Steuerung und 3D-Visualisierung.

Abbildung 3 visualisiert das Zusammenspiel der einzelnen Softwarekomponenten innerhalb des Frameworks. Dabei wird deutlich, wie die verschiedenen Technologien von der Serverbereitstellung über die Paketverwaltung bis hin zur 3D-Visualisierung und interaktiven Steuerung ineinandergreifen, um eine stabile, plattformunabhängige und benutzerfreundliche Entwicklungsumgebung bereitzustellen.

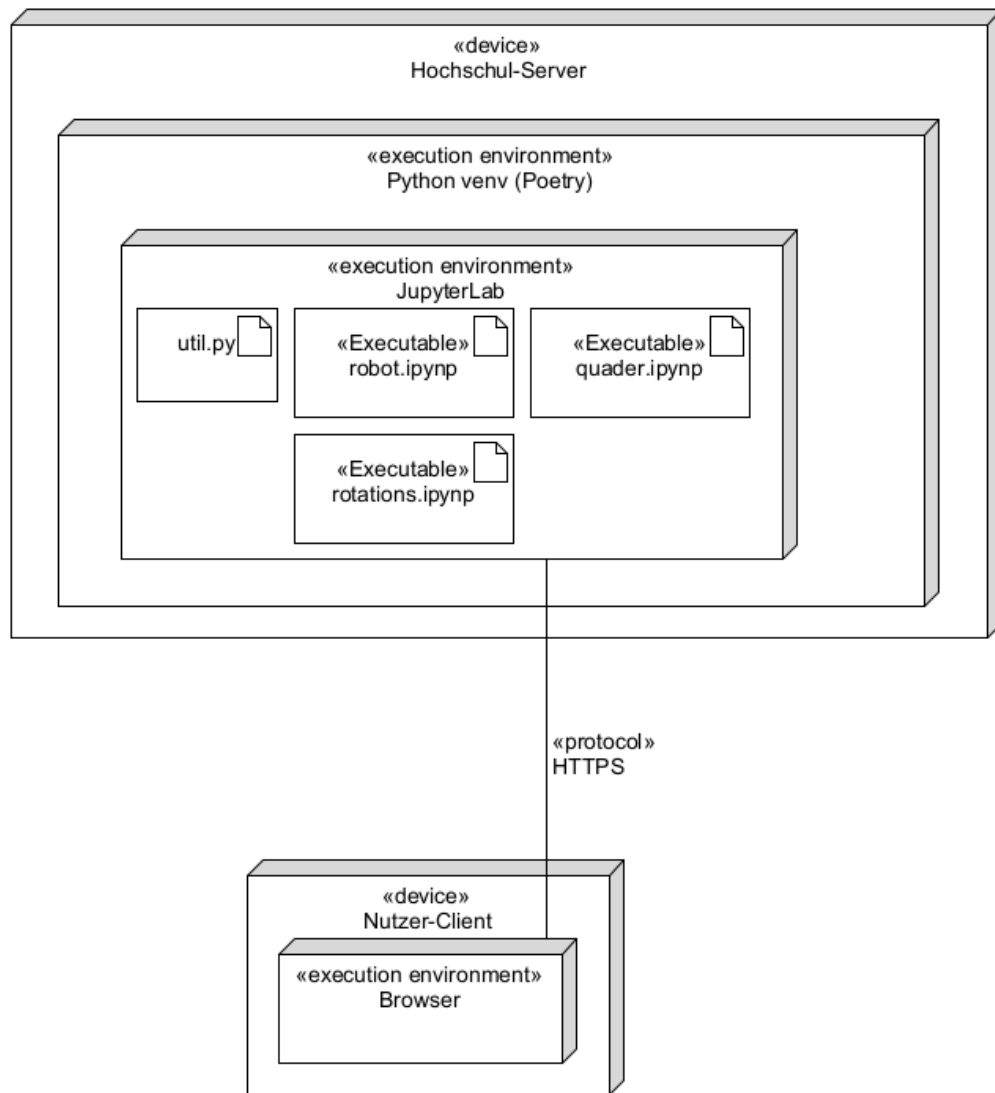


Abbildung 3: Verteilungsdiagramm des Frameworks

Das Framework ist, wie gefordert, für die Nutzung innerhalb der JupyterLab-Umgebung ausgelegt. Die Fachhochschule Dortmund stellt hierfür einen eigenen JupyterLab-Server bereit, auf den sowohl über eine VPN-Verbindung als auch direkt im Hochschulnetz zugegriffen werden kann. Dadurch ist keine lokale Installation erforderlich und die Arbeitsumgebung steht einheitlich auf verschiedenen Endgeräten zur Verfügung.

Um eine einfache Einrichtung des Frameworks auf dem Server zu gewährleisten, ist es erforderlich, Versionskonflikte zwischen den verwendeten Abhängigkeiten zu vermeiden. Zu diesem Zweck wird das Werkzeug Poetry eingesetzt. Poetry ermöglicht die Verwaltung aller benötigten Python-Bibliotheken in festen Versionen und erstellt eine isolierte virtuelle Umgebung, in der alle Abhängigkeiten in passenden Versionen zu-

einander installiert sind. Dadurch wird sichergestellt, dass das Framework unabhängig von der Zielumgebung stabil und reproduzierbar ausgeführt werden kann.

Die Bibliothek `ipywidgets` wurde verwendet, um das Framework interaktiv zu gestalten. Mithilfe von Schiebereglern, Dropdown-Menüs und Checkboxes lassen sich Rotationen der gerenderten Objekte in *weicher Echtzeit*⁶ anpassen. Darüber hinaus ermöglichen die interaktiven Elemente eine benutzerfreundliche Steuerung weiterer Funktionen und tragen wesentlich zur intuitiven Bedienbarkeit des Frameworks bei.

Eine der zentralen Bibliotheken innerhalb des Projekts ist NumPy, eine weit verbreitete Python-Bibliothek für numerische Berechnungen. NumPy wird im Kontext dieses Projekts hauptsächlich für erweiterte mathematische Operationen im Bereich der linearen Algebra eingesetzt. Außerdem ist NumPy mit der Bibliothek SymPy, die für symbolische Mathematik verwendet wird, gut kompatibel. Diese Kompatibilität ist ebenfalls eine Anforderung in diesem Projekt. Obwohl NumPy eine leistungsfähige Grundlage für numerische Berechnungen bietet, enthält es keine Funktionen für spezielle geometrische Transformationen wie beispielsweise die Umwandlung von Euler-Winkeln in Quaternionen, von Quaternionen in Rotationsmatrizen oder deren Umkehrung. Daher wurde ergänzend die Bibliothek `scipy.spatial` verwendet, die solche Operationen effizient bereitstellt.

Darüber hinaus ist Pythreejs als weitere wichtige Abhängigkeit integriert. Es handelt sich hierbei um eine Python-Wrapper-Bibliothek für die JavaScript-Bibliothek `three.js`, die auf WebGL basiert. Pythreejs ermöglicht die Darstellung und Interaktion mit 3D-Geometrien innerhalb eines WebGL-Renderers. Hierbei wird mithilfe von Pythreejs JavaScript-Code generiert und im Hintergrund automatisch auf dem Client des Nutzers ausgeführt. Die Visualisierung erfolgt somit lokal im Webbrowser innerhalb des Notebooks, wo auch die Steuerung über Python stattfindet. Der Renderer kann daher nahtlos in der JupyterLab-Umgebung eingebettet und somit direkt in einem Jupyter Notebook visualisiert und gesteuert werden. Außerdem können auf diese Weise Geometrien mit Grafikbeschleunigung gerendert werden.

Durch den Einsatz dieser Technologien wird eine effiziente und plattformunab-

⁶Weiche Echtzeit bezeichnet Systeme, bei denen Zeitvorgaben wie Bildrate ≥ 60 Hz möglichst eingehalten werden sollen, aber gelegentliche Überschreitungen tolerierbar sind und nicht zu einem Systemversagen führen.

hängige Darstellung sowie Manipulation komplexer geometrischer Strukturen innerhalb des Frameworks ermöglicht.

4.2 Framework-Library (Implementierungsdetails)

Die Bibliothek des Frameworks besteht aus einer zentralen Datei: `util.py`. In dieser Datei sind sämtliche Kernfunktionen implementiert, die zur Erstellung, Transformation und Animation geometrischer Formen benötigt werden. Darüber hinaus enthält `util.py` eine Reihe von Hilfsfunktionen, die unter anderem die Umrechnung zwischen verschiedenen Darstellungsformen von Rotationen ermöglichen sowie die Erstellung von Koordinatensystemen unterstützen.

Ergänzend dazu stellt die Bibliothek eine Klasse namens `Environment` bereit. `Environment` repräsentiert den Renderer mit einer Szene, die eine virtuelle Kamera, ein Punktlicht, ein Standardkoordinatensystem sowie ein Gitter enthält. Ein Objekt dieser Klasse kann vom Nutzer individuell angepasst werden. So lässt sich beispielsweise festlegen, ob die z-Achse oder die y-Achse nach oben zeigt. Auch das Gitter kann in seiner Auflösung verändert werden, indem die Größe der einzelnen Zellen angepasst wird. Das Gitter kann somit feiner oder gröber dargestellt werden.

Zur Darstellung innerhalb eines Jupyter Notebooks kann ein `Environment`-Objekt über die dort übliche `display()`-Funktion gerendert werden. Darüber hinaus besteht die Möglichkeit, interaktive *Widgets*⁷ für beliebige Objekte hinzuzufügen, zum Beispiel zur Steuerung der Rotation über Schieberegler. Diese Widgets werden unterhalb des Renderers angezeigt und ermöglichen eine benutzerfreundliche Interaktion mit der Szene.

Zusätzlich enthält die Bibliothek eine Funktion `move(robot, vel, steps)`, mit der die Bewegung eines radgetriebenen Roboters in einer Ebene simuliert werden kann. Die Funktion nimmt als Parameter ein Roboterobjekt, eine Geschwindigkeitsangabe sowie die Anzahl der Zeitschritte entgegen und berechnet daraus die neue Position des Roboters im Raum.

⁷Widgets sind grafische Bedienelemente in einer Benutzeroberfläche, wie z. B. Buttons, Schieberegler oder Textfelder, die der Interaktion mit dem Nutzer dienen.

4.3 Beispiel-Notebooks

Das Framework enthält drei Beispiel-Notebooks, die die Anwendungsmöglichkeiten und Funktionalität praxisnah demonstrieren. Sie sollen dem Aufgabensteller als Orientierung dienen um mithilfe des Frameworks Aufgaben oder Umgebungen zum Lernen zu generieren.

4.3.1 robot.ipynb

Eines dieser Notebooks trägt den Namen `robot.ipynb` und dient der Veranschaulichung einer einfachen Steuerungsaufgabe. Abbildung 4 zeigt die Ausgabe des Notebooks während der Ausführung.

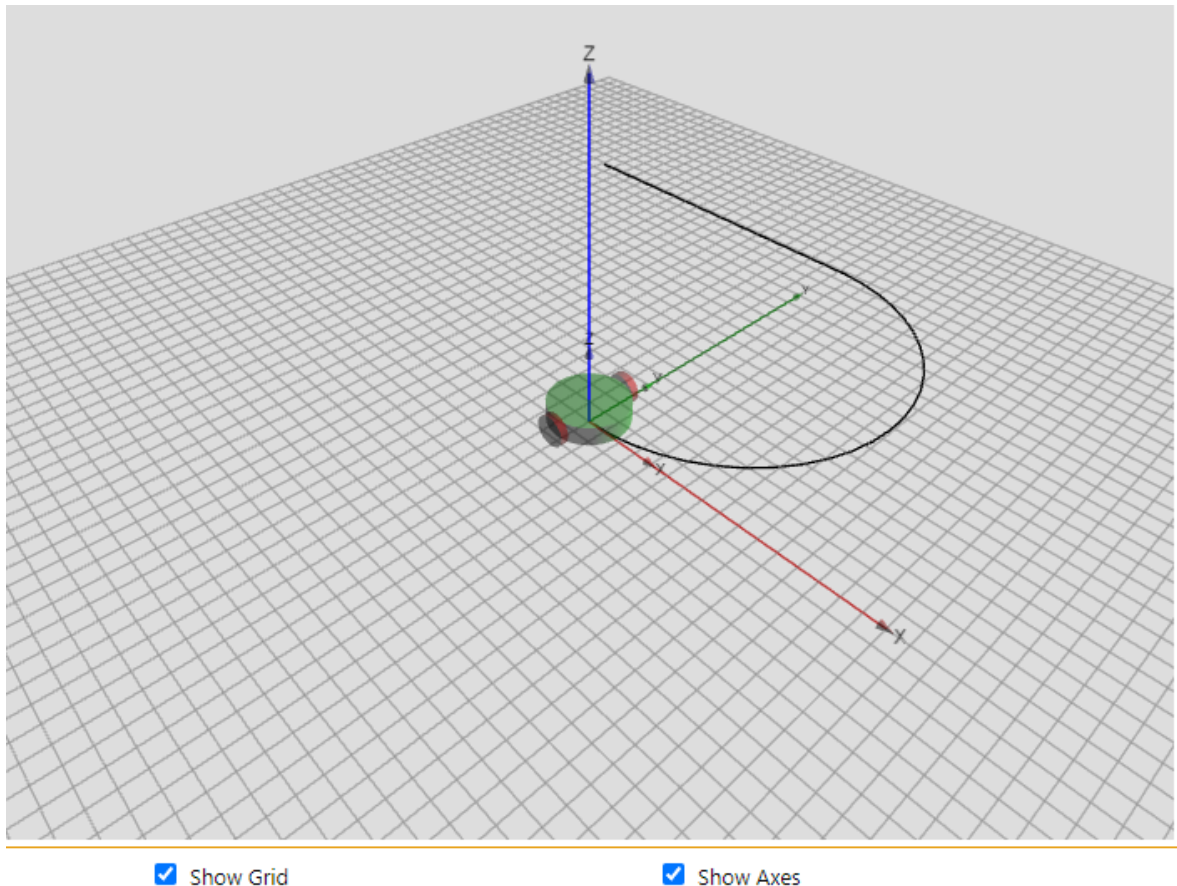


Abbildung 4: Roboter mit Differentialantrieb (`robot.ipynb` während der Ausführung)

In diesem Notebook werden zunächst in einer Codezelle ein Gitter sowie ein Koordinatensystem erstellt und zu einem Environment-Objekt hinzugefügt. Anschließend

wird ein Roboter mit Differentialantrieb erstellt sowie zwei Linien, die einen vorgegebenen Pfad repräsentieren, den der Roboter abfahren soll. Zum Erstellen des Pfads wird ein Dummy-Objekt benötigt, welches die Bewegung des Roboters für die Pfadgenerierung in der Funktion `line_wheel_driven_robot(dummy, vel, steps)` (siehe Anhang S.6) simuliert. Der Roboter und der Pfad werden zu dem Environment-Objekt hinzugefügt, welches danach mit der `display()` Funktion visuell dargestellt wird.

```
grid_group = util.create_grid_XY(32,0.5)
axes_group = util.create_axes(8)
env = util.Environment(frame=axes_group, grid=grid_group)
robot = util.create_differential_robot()

dummy = three.Group()
l1 = util.line_wheel_driven_robot(dummy, [0.01, 0, 0.00202], 1555)
l2 = util.line_wheel_driven_robot(dummy, [0.01, 0, 0.0], 990)

env.add([robot, l1, l2])
display(env)
```

Nach Ausführung dieser Zelle wird dem Benutzer die interaktive Szene wie in Abbildung 4 dargestellt präsentiert. Die Aufgabe besteht darin, die entsprechenden Antriebsgeschwindigkeiten zu berechnen und anschließend die Funktion `move(robot, vel, steps)` mit diesen Werten aufzurufen. Das Notebook orientiert sich an einer Übungsaufgabe der Fachhochschule Dortmund im Modul Robotik aus dem Jahr 2024

Gegeben ist ein mobiler Roboter mit Differentialantrieb, der den im Weltkoordinatensystem abgebildeten Kurs von A nach C vorwärts abfährt. Der Kurs besteht aus einem Halbkreis zwischen den Punkten $A = (4, 9)$ und $B = (11, 2)$ und zwischen B und $C = (18, 9)$ aus einer Geraden. Die Positionen der Punkte sind in m angegeben.

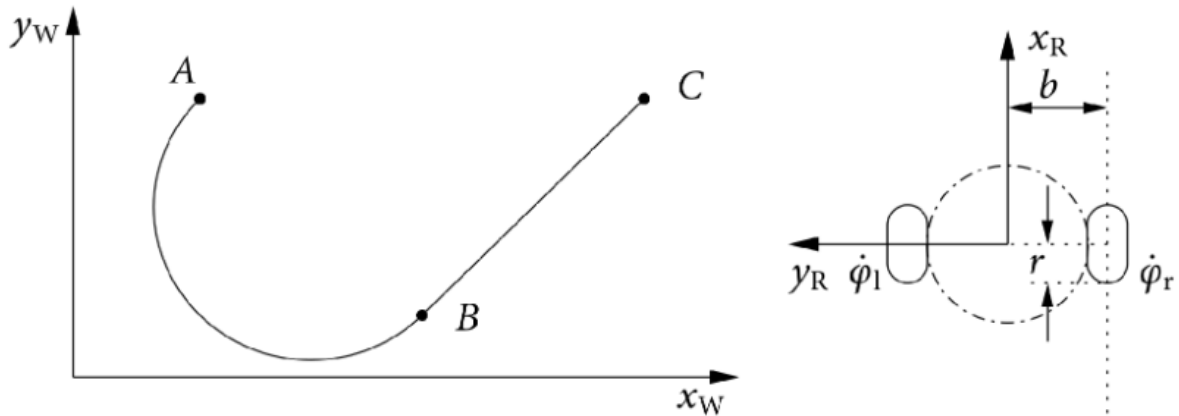


Abbildung 5: Roboter mit Differentialantrieb (Bild aus der Übungsaufgabe)

Der Roboter soll den gesamten Kurs mit einer konstanten Lineargeschwindigkeit von $\dot{x}_R = 1 \frac{m}{s}$ abfahren. Beschleunigungs- und Verzögerungsvorgänge sollen vernachlässigt werden. Der Roboter hat einen halben Radabstand von $b = 0.5m$ und Radradius von $0,25m$.

[...]

Welche Geschwindigkeiten in Roboterkoordinaten ($v_R = \dot{x}_R = (\dot{x}_R, \dot{y}_R, \dot{\theta})^T$) ergeben sich für die beiden Abschnitte jeweils? [...]

Quelle: Die Aufgabe stammt aus den eigenen Unterlagen des Moduls Robotik und ist als Datei 05_Übung_und_Praktikum.ipynb dieser Arbeit beigefügt.

In der nächsten Codezelle wird die Pose des Roboters zurückgesetzt damit wiederholte Aufrufe dieser Zelle möglich sind. Anschließend wird die `move()` Funktion für jeden Abschnitt der Bahn aufgerufen. Dabei werden in diesem Beispiel-Notebook bereits die korrekten Werte für die Antriebsgeschwindigkeiten an `move()` übergeben, was vom Aufgabensteller geändert werden kann:

```
util.set_rotation(robot, [0,0,0], "XYZ")
util.set_translation(robot, [0,0,0])

util.move(robot, [0.01, 0.0, 0.00202], 1555)
```

```
util.move(robot, [0.01, 0.0, 0.0], 990)
```

Nach Ausführung der Zelle beginnt der Roboter mit der Bewegung. Die Nutzerin bzw. der Nutzer kann dabei direkt im Jupyter Notebook beobachten, ob der Roboter dem gewünschten Pfad korrekt folgt.

4.3.2 rotations.ipynb

Ein weiteres Beispiel-Notebook trägt den Namen rotations.ipynb. In diesem Notebook wird ein dreidimensionaler Würfel visualisiert, der vom Nutzer interaktiv transformiert werden kann. Abbildung 6 illustriert das Notebook rotation.ipynb während der Ausführung und veranschaulicht die Sicht des Nutzers.

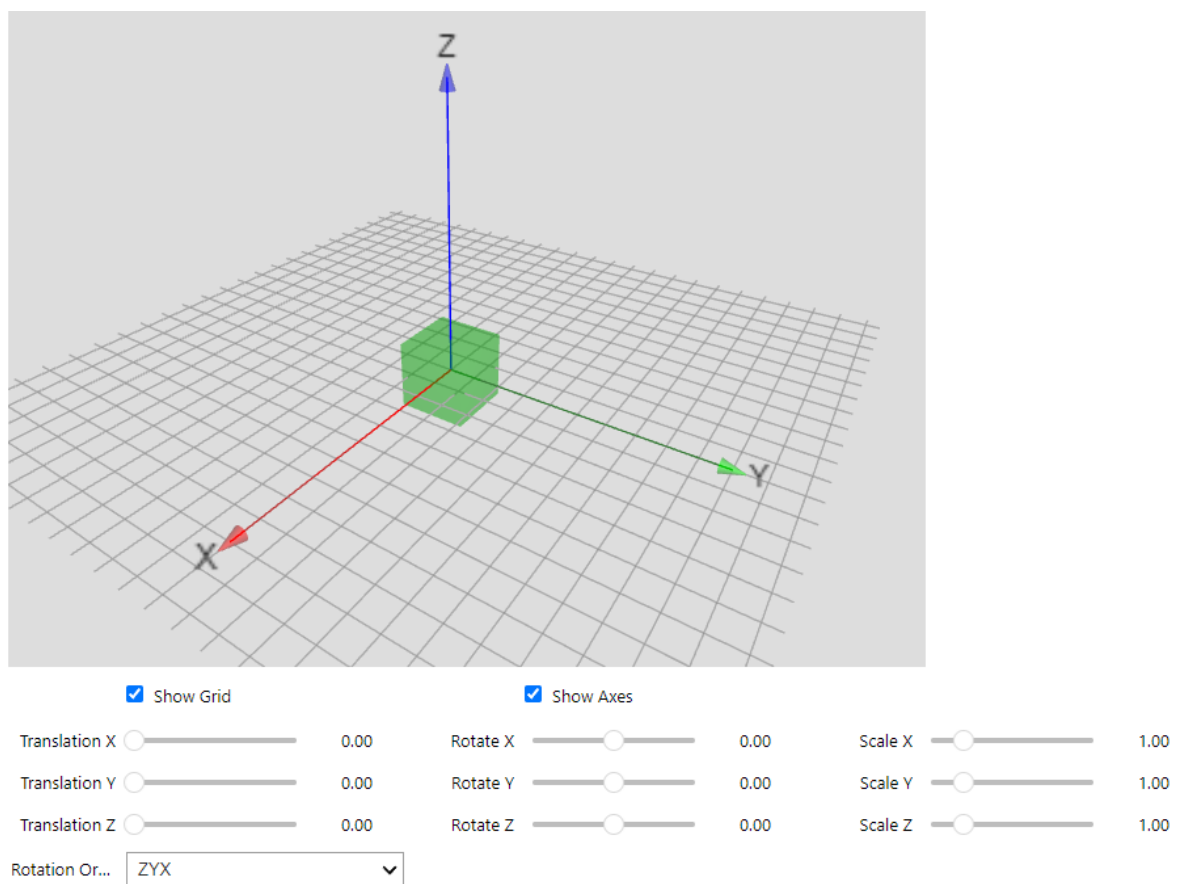


Abbildung 6: Würfel mit Schieberegler für die Transformation (rotation.ipynb während der Ausführung)

Über integrierte Schieberegler lassen sich Translation, Rotation und Skalierung des

Würfels in Echtzeit anpassen. Ziel dieses Notebooks ist es, die Möglichkeiten des Frameworks zur Darstellung und Simulation geometrischer Transformationen zu veranschaulichen. Die direkte visuelle Rückmeldung innerhalb des Jupyter Notebooks ermöglicht es dem Lernenden, die Auswirkungen der jeweiligen Transformationen unmittelbar nachzuvollziehen und ein besseres Verständnis für deren Wirkung im dreidimensionalen Raum zu entwickeln.

Um diese Ausgabe zu erreichen wurde ein Würfel sowie ein Environment-Objekt erzeugt. Der Würfel wurde zu dem Environment-Objekt hinzugefügt. Anschließend wurden mit der Funktion `add_gizmo_controls()` (siehe Anhang S.16) Schieberegler zur Transformation des Würfels zum Environment-Objekt hinzugefügt. Zum Schluss wurde das Environment-Objekt mit der `display()` Funktion visualisiert:

```
cube = util.createQuad([0,0,0],1,1,1)
env = util.Environment(
    grid=util.create_grid_XY(10,0.4), frame=util.create_axes(4))
env.camera.position=[3,3,3]
env.add(cube)
env.add_gizmo_controls(cube)

display(env)
```

4.3.3 quader.ipynb

Das letzte Beispiel-Notebook trägt den Namen `quader.ipynb`. Es orientiert sich an einer Klausuraufgabe der Fachhochschule Dortmund im Modul Robotik aus dem Jahr 2010. In der Aufgabe soll ein Quader um die Achsen des Basis-Koordinatensystems rotiert werden. Anschließend sollen die jeweiligen Rotationsmatrizen aufgestellt und zu einer Gesamtrotationsmatrix zusammengefasst werden.

Gegeben ist ein quaderförmiger Körper mit den Kantenlängen $l_x = 3, l_y = 4$ und $l_z = 2$. Eine Quaderecke befindet sich, wie in der nachfolgenden Abbildung dargestellt, im Ursprung eines ortsfesten Koordinatensystems B . In der Quaderecke K ist ein körperfestes Koordinatensystem K angebracht.

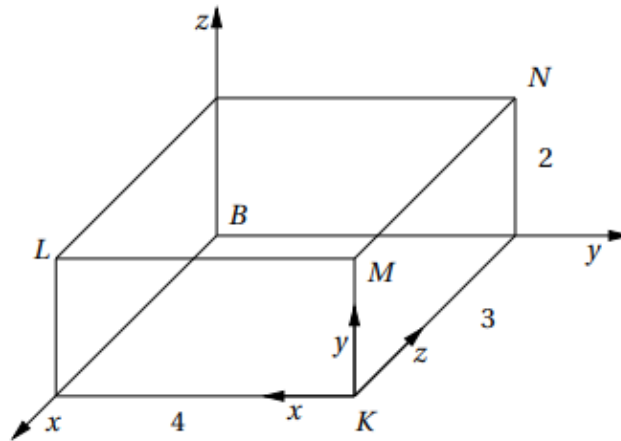


Abbildung 7: Quader aus der Klausuraufgabe

Der Körper wird nun zunächst um den Winkel $\phi_z = 60^\circ$ um die z -Achse, dann um den Winkel $\phi_x = -180^\circ$ um die x -Achse und schließlich um den Winkel $\phi_y = -90^\circ$ um die y -Achse des ortsfesten Koordinatensystems B gedreht.

a) Stellen Sie die Rotationsmatrizen $R_x := R(x, -180^\circ)$, $R_y := R(y, -90^\circ)$ und $R_z := R(z, 60^\circ)$ auf.

b) Berechnen Sie die Rotationsmatrix R_G für die Gesamttrotation.

c) Welche Positionen nehmen nach der Drehung des Körpers die Körperecken M und N in Bezug auf das Koordinatensystem B ein?

[...]

Quelle: Klausuraufgabe der Fachhochschule Dortmund im Modul Robotik aus dem Jahr 2010. Ist als Datei `RobotikWS10.pdf` dieser Arbeit beigelegt.

Das Notebook dient der visuellen Simulation der Rotationen. Abbildung 6 zeigt das Notebook `quader.ipynb` während der Ausführung.

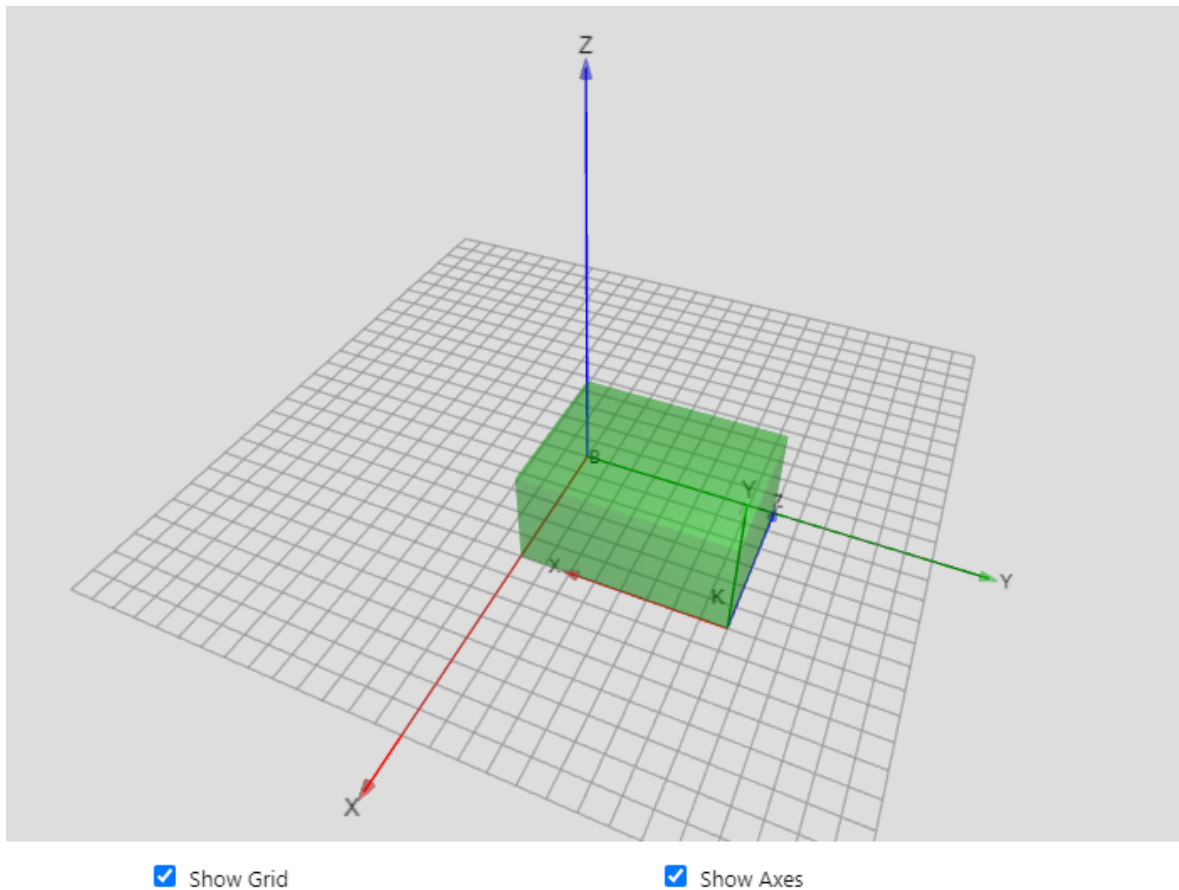


Abbildung 8: Quader aus der Klausuraufgabe (robot.ipynb während der Ausführung)

Mit der Funktion `rotate_global_animated(mesh, angles, order)` (siehe Anhang S.11) wird der Quader zunächst wie in der Aufgabe beschrieben um die Achsen des Koordinatensystems B rotiert. Diese Transformation wird visuell dargestellt, um ein Gefühl für die Rotation zu vermitteln und die Endposition aufzuzeigen. Danach wird mit der Funktion `rotate_animated(mesh, angles, order)` (siehe Anhang S.10) der Quader um sein körpereigenes Koordinatensystem rotiert, wobei jedoch die Rotationsreihenfolge invertiert wird. Diese beiden Rotationen führen den Quader in die gleiche Endposition und sollen den Lernenden diesen Zusammenhang visuell näherbringen:

```
angles=[60, -180, -90]
order = "ZXY"
util.set_rotation(b, [0,0,0], order)
util.rotate_global_animated(b, angles, order)
```

```
util.set_rotation(b, [0,0,0], order)
util.rotate_animated(b, angles[::-1], order[::-1])
```

Anschließend werden die Matrizen als Variablen aufgestellt, zusammengefasst und an die Funktion `apply_rot_matrix_animated(mesh, rot_mat, speed, show_rot_mat)` (siehe Anhang S.1) übergeben, sodass die Rotation des Quaders anhand der Rotationsmatrix betrachtet und mit der vorangegangenen Transformation verglichen werden kann:

```
rot_mat_x = np.array([[1, 0, 0],
                      [0, np.cos(np.radians(-180)), -np.sin(np.radians(-180))],
                      [0, np.sin(np.radians(-180)), np.cos(np.radians(-180))]]
                      )
```

```
rot_mat_y = np.array([
    [np.cos(np.radians(-90)), 0, np.sin(np.radians(-90))],
    [0, 1, 0],
    [-np.sin(np.radians(-90)), 0, np.cos(np.radians(-90))]]
    )
```

```
rot_mat_z = np.array([
    [np.cos(np.radians(60)), -np.sin(np.radians(60)), 0],
    [np.sin(np.radians(60)), np.cos(np.radians(60)), 0],
    [0, 0, 1]
    ])
```

```
rot_mat_ges = rot_mat_y @ rot_mat_x @ rot_mat_z
```

```
util.set_rotation(b, [0,0,0], order)
util.apply_rot_matrix_animated(b, rot_mat_ges)
```

Dieses Notebook kann vom Aufgabensteller so modifiziert werden, dass die Matrizen von den Lernenden ausgefüllt werden müssen.

5 Entwicklungsprozess

Es wurden verschiedene Technologien zur 3D-Darstellung verglichen, darunter OpenGL, WebGL, Babylon und Three.js. Nach der Erstellung von Prototypen mit Three.js und Pythreejs fiel die Entscheidung, NumPy für die mathematischen Berechnungen zu verwenden. Daraufhin wurde eine Animationslogik entwickelt, um die Bewegungen im Projekt darzustellen. Schließlich wurde die Klasse Environment entworfen, um den Code wiederverwendbar und verständlicher zu gestalten.

5.1 Wahl der Rendering-API

Zu Beginn des Entwicklungsprozesses wurden verschiedene Technologien zur Darstellung und Manipulation von 3D-Grafiken miteinander verglichen. Dabei standen OpenGL, WebGL, Babylon und Three.js im Fokus. Jede dieser Technologien bringt spezifische Vor- und Nachteile mit sich, die im Hinblick auf die Anforderungen des Projekts abgewogen wurden. Anschließend wurde ein Prototyp auf Basis von Three.js erstellt und einer auf Basis von Pythreejs um die Eignung dieser Tools für das Projekt zu validieren.

5.1.1 OpenGL

OpenGL überzeugte durch seine Plattformunabhängigkeit, da es auf verschiedenen Betriebssystemen wie Windows, macOS und Linux eingesetzt werden kann. Dies ermöglicht Entwickler:innen, grafisch anspruchsvolle Anwendungen zu erstellen, die auf unterschiedlichen Systemen laufen, ohne dass spezifische Anpassungen nötig sind. Es war besonders beliebt in Bereichen wie der 3D-Grafik, CAD-Software und der Spieleentwicklung. Jedoch bringt OpenGL auch einige Einschränkungen mit sich. Eine davon ist, dass OpenGL lokal auf dem Gerät installiert werden muss, was zusätzliche Anforderungen an die Systemkonfiguration stellt und potenziell zu Kompatibilitätsproblemen führen kann. Zudem benötigt es ein eigenes Anwendungsfenster, was die Integration in Webanwendungen erheblich erschwert, da moderne Webanwendungen typischerweise ohne lokale Installation auskommen wollen (Khronos-Group (2025b)).

5.1.2 WebGL

WebGL stellt eine moderne, browserbasierte Alternative zu OpenGL dar, die auf denselben grundlegenden Prinzipien von OpenGL basiert. WebGL läuft direkt in aktuellen Webbrowsern, ohne dass zusätzliche Software installiert werden muss, was einen entscheidenden Vorteil hinsichtlich der Zugänglichkeit bietet. Durch die Nutzung von WebGL können Entwickler:innen 3D-Inhalte in Webseiten einbinden, was besonders für interaktive Webanwendungen von Bedeutung ist. WebGL bietet ebenfalls eine plattformunabhängige Lösung, da es in allen gängigen Webbrowsern läuft, die HTML5 unterstützen. Allerdings ist WebGL relativ generisch und abstrakt aufgebaut. Dies bedeutet, dass viele der für eine vollständige 3D-Anwendung notwendigen Funktionen manuell implementiert werden müssen, was den Entwicklungsaufwand erheblich steigern kann. Zudem erfordert WebGL die Darstellung in einem separaten Browser-Tab, was die Integration von 3D-Grafiken in eine bestehende Webanwendung erschwert (Khronos-Group (2025a)).

5.1.3 Babylon.js

Babylon.js ist eine leistungsstarke 3D-Engine, die auf WebGL aufbaut und eine höher abstrahierte, fertige Lösung für die Entwicklung von 3D-Anwendungen bietet. Babylon.js ermöglicht die schnelle Erstellung komplexer 3D-Szenen, ohne dass tiefgehende Kenntnisse über das zugrunde liegende Rendering-System notwendig sind. Dies macht Babylon zu einer guten Wahl für Entwickler, die schnell funktionale 3D-Anwendungen erstellen möchten. Jedoch kann diese umfangreiche Funktionalität auch nachteilig sein, insbesondere wenn nur bestimmte Teilaspekte einer 3D-Anwendung benötigt werden. Babylon enthält beispielsweise ein vollständiges Physiksystem sowie ein Soundsystem, die für dieses Projekt nicht erforderlich sind und unnötige Komplexität hinzufügen. Ein weiterer Nachteil von Babylon.js ist, dass es auch einen eigenen Browser-Tab benötigt, um die Anwendung darzustellen, was die Benutzererfahrung und Integration in Webanwendungen einschränkt (Project-BabylonJS (2025a), Project-BabylonJS (2025c) und Project-BabylonJS (2025b)).

5.1.4 Three.js

Three.js, ebenfalls auf WebGL basierend, bietet eine abstrahierte Schnittstelle zur Erstellung von 3D-Inhalten, geht dabei jedoch nicht so weit wie Babylon und bietet daher eine größere Flexibilität und Kontrolle. Three.js liegt etwas näher an der Low-Level-Programmierung und ermöglicht es Entwickler:innen, die Details des Renderings anzupassen und zu optimieren. Dies macht Three.js zu einem guten Kompromiss zwischen einfacher Nutzung und individueller Erweiterbarkeit, da es eine gute Balance zwischen abstrahierter Funktionalität und den Möglichkeiten zur Anpassung von 3D-Szenen bietet. Besonders für ein Projekt wie dieses, das im Rahmen einer Bachelorarbeit erweitert wird, erscheint diese Flexibilität von Vorteil, da noch nicht klar ist, welche spezifischen Features in die Anwendung aufgenommen werden sollen. Im Gegensatz zu Babylon ist Three.js leichtgewichtig und erlaubt eine genauere Anpassung an die Projektanforderungen. Im Vergleich zu OpenGL oder WebGL, die eine Herangehensweise mit niedrigem Abstraktionsniveau bieten, überzeugt Three.js durch ein höheres Abstraktionsniveau, was die Entwicklung beschleunigt und vereinfacht. Wie bei den anderen Lösungen erfolgt die Darstellung auch in einem eigenen Browser-Tab, was eine ähnliche Einschränkung wie bei WebGL und Babylon darstellt. Dennoch bietet Three.js aufgrund seiner Flexibilität und einfacheren Handhabung die beste Grundlage für dieses Projekt (Project-ThreeJS (2025)).

5.2 Three.js-Prototyp

Auf Basis dieser Überlegungen wurde im weiteren Verlauf des Entwicklungsprozesses ein einfacher Prototyp mithilfe der JavaScript-Bibliothek Three.js erstellt, um zu überprüfen, ob die grundlegenden Funktionalitäten, die zur Erfüllung der Projektanforderungen notwendig sind, in Three.js enthalten sind. Zu diesem Zweck wurde über den Node Package Manager (npm) das Paket live-server installiert, das es ermöglicht, eine HTML-Seite lokal zu hosten. Auf dieser Seite wurde der Prototyp eingebunden und interaktiv getestet. Abbildung 9 zeigt die Softwareumgebung und Einbettung des Prototyps in Form eines Verteilungsdiagramms.

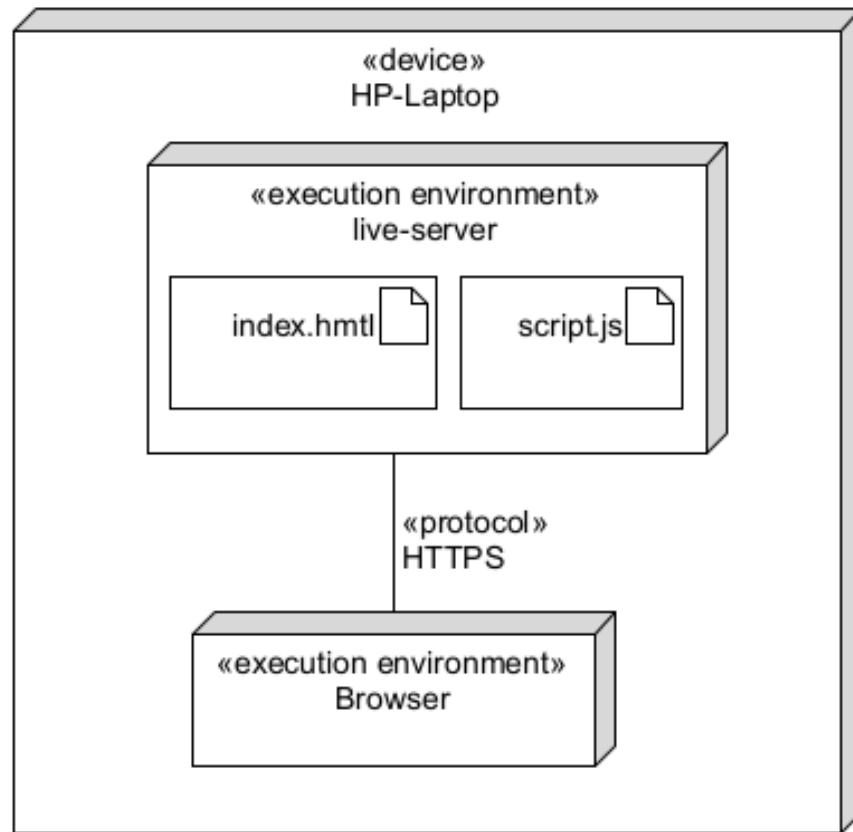


Abbildung 9: Verteilungsdiagramm des Prototyps

Wie in der Abbildung zu sehen ist, kann auf den Prototyp über den Browser zugegriffen werden, was die Portabilität sicherstellt und dafür sorgt, dass auf einem potenziellen Endgerät keine zusätzliche Software installiert werden muss. Das Artefakt index.html ist die HTML-Seite, die zur inhaltlichen Darstellung des Prototypen in einem eigenen Browsertab notwendig ist. Inhaltlich umfasst Der Prototyp zunächst eine virtuelle Kamera, eine ambiente Lichtquelle sowie einen Würfel, die alle zu einer Szene hinzugefügt werden. Für diese Schritte ist das Artefakt script.js zuständig, welches von index.HTML eingebunden wird. Abbildung 10 zeigt den Prototypen während der Ausführung im Browserfenster.

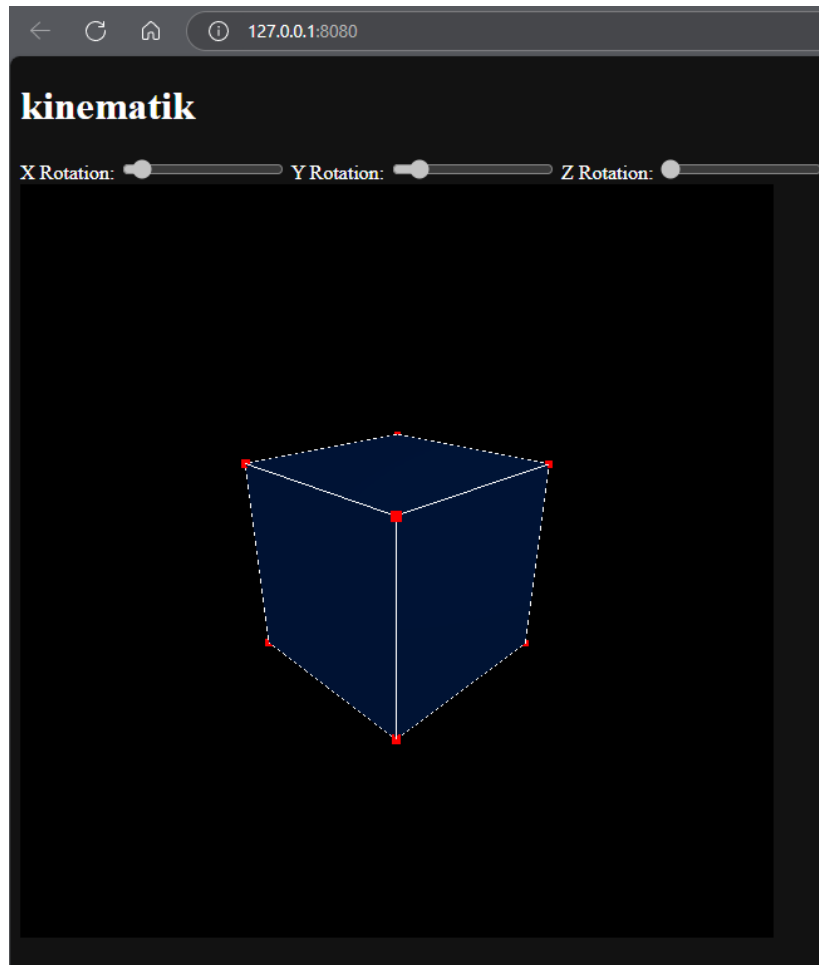


Abbildung 10: Prototyp mit Three.js

Dabei stellt der schwarze Bereich in Abbildung 10 die Szene dar. Die virtuelle Kamera stellt die Perspektive, aus der die Szene gerendert werden soll, dar und ist daher nicht als Objekt in der Abbildung zu sehen. Die ambiente Lichtquelle sorgt dafür, dass der Würfel bei der Farbberechnung seine blaue Farbe beibehält und nicht Schwarz gerendert wird und ist ebenfalls nicht als Objekt in der Szene zu sehen. Ergänzend dazu sind in der `index.html`-Datei Schieberegler integriert, mit denen die Rotation des Würfels über eine Benutzeroberfläche interaktiv gesteuert werden kann. Der Würfel wird pro Animationsframe geringfügig um die x- und y-Achse des körpereigenen Koordinatensystems rotiert. Dies stellt die grundlegenden Funktionen des Prototyps dar: Die interaktive Steuerung sowie Animation eines 3D-Objekts. Der Prototyp erfüllt damit die zentralen Anforderungen, indem er eine dynamische 3D-Visualisierung bietet, bei der der Benutzer oder die Benutzerin in Echtzeit Einfluss auf die Darstellung nehmen kann. Diese erste Version des Prototyps ist ein wichtiger Schritt, um die Funktionalität

und Flexibilität von Three.js für das Projekt zu validieren.

Ein offensichtlicher Nachteil dieses Ansatzes ist jedoch, dass die visuelle Darstellung des Würfels in einem separaten Browser-Tab läuft. Die Python-Schnittstelle, die für das eigentliche Framework vorgesehen ist, würde jedoch in der JupyterLab-Umgebung und somit ebenfalls in einem eigenen Tab ausgeführt werden. Trotz dieser Einschränkung bestätigt der Prototyp, dass Three.js die benötigten Basisfunktionalitäten bereitstellt.

5.3 Pythreejs-Prototyp

Der Nachteil, dass die visuelle Darstellung in einem separaten Browser-Tab erfolgt, lässt sich umgehen, indem man eine Python-basierte Schnittstelle verwendet, die direkt im JupyterLab-Umfeld arbeitet. Hierzu bietet sich die Bibliothek Pythreejs an, ein Wrapper für Three.js, der eine Python-API zur Verfügung stellt, welche im Hintergrund JavaScript-Code generiert und ausführt.

Pythreejs ist eine auf Jupyter Widgets basierende Notebook-Erweiterung, die es Jupyter ermöglicht, die WebGL-Fähigkeiten moderner Browser zu nutzen, indem sie *Bindings*⁸ zur JavaScript-Bibliothek Three.js bereitstellt. Durch die Einbettung in die jupyter-widgets-Infrastruktur lässt sich Pythreejs nahtlos mit anderen interaktiven Werkzeugen innerhalb von Jupyter-Notebooks kombinieren, was die Entwicklung komplexer, interaktiver Visualisierungen erheblich erleichtert (Project-Pythreejs (2025b)).

Die API von Pythreejs orientiert sich dabei stark an der von Three.js, sodass vorhandene Dokumentationen und Ressourcen weitgehend übertragbar sind. Ein wesentlicher Unterschied liegt im sogenannten Render-Loop: Während Three.js typischerweise kontinuierlich *Frames*⁹ rendert, erlaubt Pythreejs sowohl eine gezielte Einzelbilddarstellung per Aufruf als auch interaktive Darstellungen über spezielle Renderer-Klassen. Diese ermöglichen Funktionen wie OrbitControls (für das Schwenken, Zoomen und Drehen der Kamera) direkt innerhalb des Notebooks, ohne dass ein separater Browser-Tab erforderlich ist (Project-Pythreejs (2025a)).

Nach weiteren Tests wurde deutlich, dass Pythreejs im Vergleich zu Three.js einige

⁸Bindings sind Schnittstellen, die es ermöglichen, Funktionen oder Klassen aus einer Programmiersprache in einer anderen zu verwenden.

⁹Frame = Einzelnes Bild innerhalb einer Bildsequenz, das z. B. bei Animationen oder beim Rendering erzeugt und angezeigt wird.

Einschränkungen mit sich bringt. Eine zentrale Erkenntnis war, dass sich das rotation-Attribut eines *Mesh*¹⁰-Objekts in Pythreejs nicht direkt verändern lässt. Stattdessen kann das quaternion-Attribut genutzt werden, um die Rotation zu steuern. Das macht es notwendig, Hilfsfunktionen zu implementieren, die Euler-Winkel in Quaternionen umrechnen. Dieser Schritt ist jedoch unproblematisch, da solche Funktionen ohnehin aus didaktischen Gründen Teil des Frameworks sein sollen.

Auf Basis dieser Erkenntnisse wurde ein erweiterter Prototyp mit Pythreejs erstellt, bei dem die entwickelten Hilfsfunktionen aus der Datei `util.py` zum Einsatz kommen. Abbildung 11 zeigt den visuellen Output des Renderers innerhalb des Prototyps, wobei es sich bei dem Prototyp um ein Jupyter-Notebook handelt.

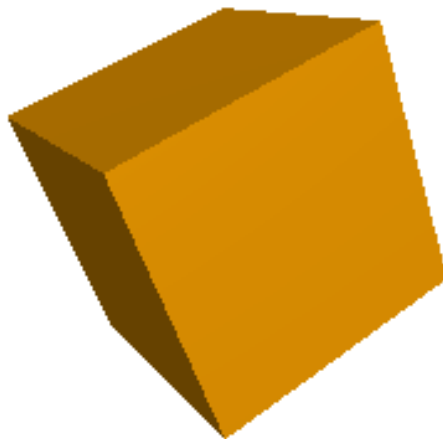


Abbildung 11: Notebook-Prototyp mit Pythreejs

Dieser Prototyp zeigte erfolgreich, dass die für das Framework essenziellen Grund-

¹⁰Engl. Begriff für ein Netz aus Eckpunkten (Vertices), Kanten und Flächen, das zur Darstellung von 3D-Geometrien verwendet wird.

funktionen – insbesondere die Animation und Rotation von Objekten – umsetzbar sind. Interaktive Elemente konnten hier nicht mehr wie im vorherigen Prototyp durch HTML-Elemente hinzugefügt werden. Dafür wurde hier die Python-Bibliothek `ipywidgets` genutzt.

5.4 Numpy als Mathematik-Library

Da der Prototyp verschiedene *numerische*¹¹ Berechnungen zur Transformation und Animation von Objekten durchführt, stellte sich die Frage, welche Bibliothek für die linearen Algebraoperationen am besten geeignet ist. Obwohl das Framework so gestaltet ist, dass es mit `sympy.Matrix`-Objekten umgehen kann, wurde in der konkreten Implementierung bewusst auf NumPy als Rechenbibliothek gesetzt. Der Grund dafür liegt in der deutlich besseren Effizienz von NumPy bei numerischen Berechnungen. Während SymPy primär auf *symbolische*¹² Mathematik ausgelegt ist, wie das Umformen, Ableiten oder Vereinfachen mathematischer Ausdrücke, ist NumPy auf die schnelle numerische Verarbeitung konkreter Zahlenwerte optimiert.

Das zeigt sich besonders deutlich in der Rechengeschwindigkeit: In einem einfachen Test zur Matrixmultiplikation zweier 100x100-Matrizen benötigte NumPy nur etwa 0.0017 Sekunden. SymPy benötigte für dieselbe Operation rund 9.7 Sekunden, wie man an der folgenden Konsolenausgabe sehen kann:

```
NumPy Zeit: 0.001711 Sekunden
SymPy Zeit: 9.699459 Sekunden
```

Der Unterschied liegt dabei nicht im mathematischen Verfahren selbst, sondern in der Art der Datenverarbeitung: NumPy arbeitet mit kompakten, typisierten Datenstrukturen und nutzt intern optimierte C-Implementierungen, während SymPy jeden Rechenschritt symbolisch verfolgt und daher deutlich langsamer ist (Project-Numpy (2025) und Project-Sympy (2025)).

¹¹Im mathematischen Sinn bedeutet numerisch, dass ein Problem mithilfe von Zahlenwerten und Näherungsverfahren gelöst wird, anstatt durch exakte symbolische Umformungen.

¹²Im mathematischen Sinn bezeichnet symbolisch die Verarbeitung von Gleichungen und Ausdrücken durch exakte algebraische Umformungen unter Verwendung von Variablen und Symbolen, im Gegensatz zu numerischen Näherungen.

Da das Ziel des Frameworks ist, Transformationen numerisch auszuwerten und direkt auf reale Objekte anzuwenden, war eine symbolische Verarbeitung nicht erforderlich. Dennoch war eine der zentralen Anforderungen, dass das Framework mit `sympy.Matrix`-Objekten kompatibel ist. Daher wurde sichergestellt, dass solche Eingaben akzeptiert und automatisch in ein numerisch geeignetes Format überführt werden. Dadruch bleibt auch bei der Verwendung von SymPy eine korrekte und performante Ausführung gewährleistet. Die Voraussetzung dafür ist jedoch, dass die übergebenen `sympy.Matrix`-Objekte sich auf konkrete Werte oder Annäherungen zurückführen lassen.

5.5 Animationslogik

Darauf aufbauend wurden die Notebooks `rotation.ipynb`, `quader.ipynb` und `robot.ipynb` entwickelt. Parallel dazu wurde die Bibliothek `util.py` schrittweise um zusätzliche Funktionen ergänzt, um komplexere Abläufe und Animationen realisieren zu können. Ein zentrales Prinzip bei der Animationslogik in `util.py` ist die Verwendung eines zeitbasierten Interpolationsschemas. Dabei wird die Rotation, Skalierung oder Translation zwischen zwei Zuständen schrittweise überführt. Der Ablauf folgt dabei einem einfachen Muster:

Algorithm 1 Interpolation einer Transformation

1: <i>speed</i>	▷ Interpolationsgeschwindigkeit
2: $t \leftarrow 0$	
3: $\Delta t \leftarrow 0.002$	
4: while $t \leq 1$ do	
5: $mesh.transform \leftarrow interpolate(old_transform, new_transform, t)$	
6: $t \leftarrow t + \Delta t$	
7: $wait(1/speed)$	
8: end while	
9: $mesh.transform \leftarrow interpolate(old_transform, new_transform, 1)$	

Dabei beschreibt t den Fortschritt der Interpolation von 0 bis 1, Δt ist die Schrittweite, welche einen kleinen Wert haben sollte damit die Animation flüssig läuft. In jedem Schritt wird eine neue Zwischen-Rotation, Position oder Skalierung berechnet und auf das Objekt angewendet, sodass eine flüssige Bewegung entsteht. Bei der praktischen Umsetzung des Algorithmus wurde die python library `time` für das Warten verwendet. Es fiel auf, dass `time.sleep()` ab einem gewissen Schwellenwert (etwa 1ms)

keine weiteren Verkürzungen der Wartezeit bewirkt. Dies liegt vermutlich an dem overhead der Funktion. Auch wenn kleinere Werte übergeben werden, wird faktisch immer eine Mindestwartezeit eingehalten, die sich technisch nicht weiter reduzieren lässt. Dadurch war die Steuerung der Animationsgeschwindigkeit unterhalb dieser Grenze nicht mehr differenzierbar.

5.6 Environment-Klasse

Zum Ende der Entwicklungsarbeit hin wurde klar, welche Gemeinsamkeiten die Szenarien in `rotation.ipynb`, `quader.ipynb` und `robot.ipynb` und womöglich auch zukünftige Aufgabentypen haben. So wurde die Klasse `Environment` entworfen, die wiederkehrende Elemente in einer flexiblen und erweiterbaren Struktur zusammenfasst.

Die Klasse `Environment` kapselt die Erstellung und Verwaltung einer dreidimensionalen Szene. Dabei werden grundlegende Bestandteile wie Kamera, Lichtquellen, Gitter und Achsenobjekte automatisch erzeugt und zu einer `Scene`-Instanz zusammengefügt. Darüber hinaus integriert die Klasse ein interaktives `Renderer`-Element mit Kamera-Steuerung (`OrbitControls`), das direkt im Notebook angezeigt wird.

Um eine besonders einfache Erweiterbarkeit zu gewährleisten, wurden Methoden wie `add`, `add_widget` und `add_gizmo_controls` hinzugefügt, die beliebige Objekte und interaktive Steuerelemente zur Szene hinzufügen.

Die `Environment`-Klasse übernimmt somit eine zentrale Rolle innerhalb des Frameworks. Sie abstrahiert technische Details der Visualisierung und erlaubt es, sich auf die inhaltliche Modellierung von Bewegungsabläufen zu konzentrieren. Dadurch soll die Entwicklung und Darstellung von Aufgaben in den Notebooks `rotation.ipynb`, `quader.ipynb` und `robot.ipynb` vereinfacht werden.

Die Kombination aus `pythreejs`, den in `util.py` enthaltenen Transformationsfunktionen und der `Environment`-Klasse bildet somit ein Fundament für die weitere Entwicklung des Frameworks. Erweiterungen wie die Entwicklung aufgabenbezogener Rotationszenarien oder die schrittweise Analyse komplexerer Bewegungsabläufe lassen sich auf dieser Basis modular und interaktiv umsetzen.

6 Evaluation

Im Rahmen dieses Projekts wurden viele funktionale und technische Anforderungen an das zu entwickelnde Visualisierungs-Framework definiert. Bei genauer Betrachtung lässt sich sagen, dass die Anforderungen erfüllt wurden.

Eine dieser Anforderungen war, das Framework so zu entwickeln, dass es direkt in JupyterLab genutzt werden kann. Die Anwendung lässt sich problemlos in Jupyter Notebooks einbinden, sodass diese Anforderung erfüllt worden ist.

Ein weiteres Kriterium war die interaktive Bedienbarkeit. Durch die Verwendung von ipywidgets konnten benutzerfreundliche Bedienelemente wie Schieberegler und Dropdown-Menüs implementiert werden. Diese ermöglichen eine intuitive Steuerung von Parametern, wie zur Anpassung von Rotationen und erfüllen somit die Anforderung nach einer interaktiven Auseinandersetzung mit kinematischen Konzepten.

Die Kompatibilität mit SymPy wurde ebenfalls sichergestellt. Die Funktionen in Util.py können Matrixobjekte der Sympy Bibliothek entgegennehmen, sofern die Symbole sich auf konkrete Werte zurückführen lassen.

Im inhaltlichen Fokus stand die Darstellung und Transformation von Rotationen im dreidimensionalen Raum. Die Umsetzung verschiedener Darstellungsformen wie Eulerwinkel, Roll-Pitch-Yaw-Winkel, Rotationsmatrizen und Quaternionen wurde realisiert. Körper können in dem Framework anhand jeder dieser Rotationendarstellungen transformiert werden. Die entsprechenden Umrechnungen zwischen diesen Darstellungen wird durch Funktionen der Library util.py ermöglicht.

Auch die Simulation von radgetriebenen Robotern wurde integriert. Mit der Funktion `move(robot, vel, steps)` lassen sich einfache Bewegungsabläufe in der Ebene darstellen. Das erweitert das Framework über reine Rotationen hinaus.

Die Verwendung von Poetry zur Verwaltung von Abhängigkeiten sorgt darüber hinaus für eine stabile und konsistente Entwicklungsumgebung, was insbesondere für die langfristige Nutzung und Weiterentwicklung des Frameworks relevant ist.

Ein Aspekt, der im aktuellen Projektstatus noch nicht vollständig umgesetzt wurde, ist die Modellierung stationärer Roboter (z.B. Knickarmroboter) samt Vorwärts- und Rückwärtskinematik. Allerdings wurde dies von Beginn an als zukünftige Erweiterung vorgesehen und fällt nicht in den unmittelbaren Umfang dieser Projektarbeit.

Es lässt sich feststellen, dass das Framework die definierten Anforderungen erfüllt. Es bietet eine funktional umfangreiche, technisch stabile Plattform zur Unterstützung der Lehre im Bereich der robotischen Kinematik. Die Grundlage für eine spätere Erweiterung um komplexere Robotermodelle wurde ebenfalls gelegt, womit auch die langfristigen Projektziele realistisch erscheinen.

7 Ausblick auf die Bachelorarbeit

Aufbauend auf den Ergebnissen dieses Projekts ergeben sich viele Möglichkeiten zur Weiterentwicklung, die sich gut im Rahmen einer Bachelorarbeit umsetzen lassen. Ein naheliegender nächster Schritt ist es, das bestehende Framework um Funktionen zur Modellierung stationärer Roboter zu erweitern. Ein Knickarmroboter würde sich dafür besonders gut eignen.

Der Fokus dieser Erweiterung soll auf der Umsetzung der Vorwärts- und Rückwärtskinematik liegen. Während die Vorwärtskinematik beschreibt, wie man aus bekannten Gelenkwinkeln die Position des Endeffektors berechnet, geht es bei der Rückwärtskinematik darum, herauszufinden, welche Gelenkstellungen nötig sind, um eine bestimmte Zielposition zu erreichen. Beide Verfahren sind essenziell für die Steuerung und Planung von Roboterbewegungen und stellen im Vergleich zu den bisherigen Transformationen im Projekt eine sinnvolle und etwas komplexere Erweiterung dar.

Ein wichtiges Ziel der Bachelorarbeit ist es außerdem, das Framework auf dem JupyterLab-Server der Fachhochschule Dortmund bereitzustellen, sodass es für Studierende leicht zugänglich wird. Dadurch soll sichergestellt werden, dass die neuen Funktionen nicht nur theoretisch existieren, sondern auch direkt im Lehrbetrieb eingesetzt werden können. Die Idee ist, dass Studierende ohne zusätzliche Installationen direkt im Browser mit dem Framework arbeiten und damit verschiedene Robotermodelle und kinematische Aufgaben ausprobieren können.

Auch die bestehenden interaktiven Elemente sollen weiterentwickelt werden. Beispielsweise könnten bei der Rückwärtskinematik verschiedene Lösungsansätze visualisiert und direkt verglichen werden. So bleibt das Framework weiterhin intuitiv und visuell verständlich.

Die Bachelorarbeit zu diesem Thema ist eine gute Gelegenheit, das Framework

inhaltlich und funktional weiterzuentwickeln und gleichzeitig einen praktischen Beitrag zur Lehre an der Fachhochschule Dortmund zu leisten.

Literatur

- CoppeliaRobotics (2025). *Manual*. URL: <https://manual.coppeliarobotics.com/> (besucht am 05.05.2025).
- Corke, Peter (2023). *Robotics, Vision and Control: Fundamental Algorithms in Python*. eng. Third edition. Bd. 146. Springer Tracts in Advanced Robotics. Cham: Springer International Publishing AG. ISBN: 3031064682.
- Ernesti, Johannes und Peter Kaiser (2017). *Python 3 : das umfassende Handbuch*. ger. 5., aktualisierte Auflage. Rheinwerk Computing. Bonn: Rheinwerk Verlag. ISBN: 9783836258647.
- Gutiérrez, Ángel, Gilah C Leder, Paolo Boero, Paolo Boero, Ángel Gutiérrez und Gilah C. Leder (2016). *The Second Handbook of Research on the Psychology of Mathematics Education: The Journey Continues*. eng. 1. Aufl. Rotterdam: Springer. ISBN: 9463005609.
- Hertzberg, Joachim, Kai Lingemann und Andreas Nüchter (2012). *Mobile Roboter : eine Einführung aus Sicht der Informatik*. ger. eXamen.press. Berlin [u.a: Springer. ISBN: 9783642017254.
- Khronos-Group (2025a). *Getting Started*. URL: https://www.khronos.org/webgl/wiki/Getting_Started (besucht am 02.05.2025).
- (2025b). *The Industry's Foundation for High Performance Graphics*. URL: <https://www.khronos.org/opengl/> (besucht am 02.05.2025).
- Kuipers, J. B. (1999). *Quaternions and Rotation Sequences : A Primer with Applications to Orbits, Aerospace and Virtual Reality*. eng. Princeton, NJ: Princeton University Press. ISBN: 9780691211701.
- Project-BabylonJS (2025a). *BabylonJS*. URL: <https://www.babylonjs.com/> (besucht am 30.04.2025).
- (2025b). *BabylonJS Dokumentation Physics*. URL: <https://doc.babylonjs.com/features/featuresDeepDive/physics/> (besucht am 30.04.2025).
- (2025c). *BabylonJS Specification*. URL: <https://www.babylonjs.com/specifications/> (besucht am 30.04.2025).
- Project-Jupyter (2024). *JupyterLab Documentation*. URL: <https://jupyterlab.readthedocs.io/en/stable/> (besucht am 14.04.2025).

- Project-Numpy (2025). *what is numpy*. URL: <https://numpy.org/devdocs/user/whatisnumpy.html> (besucht am 03.05.2025).
- Project-Pythreejs (2025a). *Introduction*. URL: <https://pythreejs.readthedocs.io/en/stable/introduction.html> (besucht am 04.05.2025).
- (2025b). *pythreejs*. URL: <https://pythreejs.readthedocs.io/en/stable/index.html> (besucht am 04.05.2025).
- Project-Sympy (2025). *advanced usage*. URL: <https://docs.sympy.org/latest/explanation/best-practices.html#advanced-usage> (besucht am 03.05.2025).
- Project-ThreeJS (2025). *ThreeJS Manual Fundamentals*. URL: <https://threejs.org/manual/#en/fundamentals> (besucht am 30.04.2025).
- Siegwart, Roland, Illah Reza Nourbakhsh und Davide Scaramuzza (2011). *Introduction to autonomous mobile robots*. eng. 2. ed. Intelligent robotics and autonomous agents. Cambridge, Mass. [u.a: MIT Press. ISBN: 9780262015356.

8 Anhang

8.1 Code-Dokumentation der Python-Bibliothek util

Im folgenden Anhang ist die Dokumentation der Python-Bibliothek enthalten. Jede Funktion wird mit einer kurzen Beschreibung sowie dem zugehörigen Code vorgestellt. Die Dokumentation der Library util.py ist ebenfalls als HTML-Datei `util.html` dieser Arbeit beigelegt.

```
def apply_rot_matrix(mesh, rot_mat):
```

Wendet eine Rotationsmatrix auf ein Mesh-Objekt an, indem die Matrix in ein Quaternion umgewandelt wird und auf das bestehende Quaternion des Meshs angewendet wird.

Parameter:

- `mesh`: Das Mesh-Objekt, auf das die Rotation angewendet werden soll. Erwartet wird, dass das Mesh ein quaternion-Attribut besitzt.
- `rot_mat`: Die Rotationsmatrix, die auf das Mesh angewendet werden soll. Muss eine 3x3 Matrix sein.

Rückgabewert:

- Keine Rückgabe, das Mesh-Objekt wird direkt modifiziert.

```
def apply_rot_matrix_animated  
(mesh, rot_mat, speed=100, show_rot_axis=True):
```

Wendet eine Rotationsmatrix auf ein Mesh an und rotiert es animiert mit einer gegebenen Geschwindigkeit. Dabei kann optional eine Rotationsachse angezeigt werden.

Parameter:

- `mesh`: Das Mesh, das rotiert werden soll.
- `rot_mat`: Die Rotationsmatrix, die auf das Mesh angewendet werden soll.

- **speed:** Die Geschwindigkeit der Animation, angegeben als Anzahl der Frames pro Sekunde. Standardwert ist 100.
- **show_rot_axis:** Ein Boolean-Wert, der angibt, ob die Rotationsachse angezeigt werden soll. Standardwert ist `True`.

Rückgabewert:

- Keine Rückgabe (Void-Funktion).
-

```
def compute_normals(vertices, indices):
```

Generiert die Normalen für alle Dreiecke, die sich aus `vertices` und `indices` ergeben.

Parameter:

- **vertices:** Die Punkte im Raum, aus denen die Dreiecke bestehen, für die die Normalen berechnet werden, als Array.
- **indices:** Die Indizes zum Verbinden der Punkte zu Dreiecken, für welche dann die Normalen berechnet werden, als Array.

Rückgabewert:

- Ein Array, welches die Normalen enthält.
-

```
def create_axes(len, font_scale=0.4, show_labels=True, name=''):
```

Erstellt ein 3D-Koordinatensystem mit den Achsen X, Y, Z und optionalen Beschriftungen.

Parameter:

- **len:** Länge der Achsen.
- **font_scale:** Skalierung der Schriftgröße für die Achsenbeschriftungen.
- **show_labels:** Boolescher Wert, der angibt, ob die Achsenbeschriftungen angezeigt werden sollen.
- **name:** Optionaler Name, der in der Mitte des Koordinatensystems angezeigt wird.

Rückgabewert:

- Ein 3D-Objekt (Group), das das Koordinatensystem mit Achsen und optionalen Beschriftungen enthält.
-

```
def create_cylinder  
(pos, radiusTop=1, radiusBottom=1, height=2, radialSegments=32,  
color=[255, 0, 0], transparent=True):
```

Erstellt ein Zylinder-Mesh mit der angegebenen Position, Größe und Farbe.

Parameter:

- **pos**: Die Position des Zylinders als Array oder Tuple `[x, y, z]`.
- **radiusTop**: Der Radius des Zylinders an der Oberseite. Standardwert ist 1.
- **radiusBottom**: Der Radius des Zylinders an der Unterseite. Standardwert ist 1.
- **height**: Die Höhe des Zylinders. Standardwert ist 2.
- **radialSegments**: Die Anzahl der radialen Segmente des Zylinders, die die Auflösung rund um den Zylinder bestimmen. Standardwert ist 32.
- **color**: Die Farbe des Zylinders als Array `[R, G, B]`, wobei jede Komponente im Bereich 0-255 liegt. Standardwert ist `[255, 0, 0]` (Rot).
- **transparent**: Ein Boolean-Wert, der angibt, ob das Material transparent sein soll. Standardmäßig `True`.

Rückgabewert:

- Ein Mesh-Objekt, das den Zylinder darstellt, mit der angegebenen Position, Größe und Farbe.
-

```
def create_box(pos, width, height, depth, color=[0, 255, 0],
transparent=True):
```

Erzeugt ein Quader-Mesh (Box) mit der angegebenen Position, Größe und Farbe.

Parameter:

- `pos`: Die Position des Quaders als Array oder Tuple `[x, y, z]`.
- `width`: Die Breite des Quaders.
- `height`: Die Höhe des Quaders.
- `depth`: Die Tiefe des Quaders.
- `color`: Die Farbe des Quaders als Array `[R, G, B]`, wobei jede Komponente im Bereich 0-255 liegt. Standardmäßig grün `[0, 255, 0]`.
- `transparent`: Ein Boolean-Wert, der angibt, ob das Material transparent sein soll. Standardmäßig `True`.

Rückgabewert:

- Ein Mesh-Objekt, das den Quader darstellt, mit der angegebenen Position, Größe und Farbe.
-

```
def create_grid_XY(size, density):
```

Erstellt ein 3D-Gitter im XY-Plane mit der angegebenen Größe und Dichte.

Parameter:

- `size`: Die Größe des Gitters (die Ausdehnung in X- und Y-Richtung).
- `density`: Die Dichte des Gitters, die angibt, wie viele Linien innerhalb des Gitters erstellt werden.

Rückgabewert:

- Ein 3D-Objekt (Group), das das Gitter mit Linien im XY-Plane enthält.
-

```
def create_grid_XZ(size, density):
```

Erstellt ein 3D-Gitter im XZ-Plane mit der angegebenen Größe und Dichte.

Parameter:

- **size:** Die Größe des Gitters (die Ausdehnung in X- und Z-Richtung).
- **density:** Die Dichte des Gitters, die angibt, wie viele Linien innerhalb des Gitters erstellt werden.

Rückgabewert:

- Ein 3D-Objekt (Group), das das Gitter mit Linien im XZ-Plane enthält.
-

```
def create_differential_robot():
```

Erzeugt einen zylinderförmigen Roboter mit Differentialantrieb. Der Roboter hat zwei Räder.

Parameter:

- Keine Parameter.

Rückgabewert:

- Ein 3D-Objekt (Mesh) das den Differentialroboter darstellt.
-

```
def euler_to_quaternion(angles, order='YZY'):
```

Wandelt Eulerwinkel in ein Quaternion um.

Parameter:

- **angles:** Die Eulerwinkel in Grad. Diese müssen in der Reihenfolge angegeben werden, wie es **order** vorgibt, z. B. **angles=[y, x, z]**, **order="YXZ"**.
- **order:** Rotationsreihenfolge für die Eulerwinkel als String.

Rückgabewert:

- Das Quaternion als Array.
-

```
def euler_to_rot_mat(angles, order='ZYZ'):
```

Wandelt Eulerwinkel in eine Rotationsmatrix um.

Parameter:

- **angles:** Die Eulerwinkel in Grad. Diese müssen in der Reihenfolge angegeben werden, wie es **order** vorgibt, z. B. `angles=[y, x, z]`, `order="YXZ"`.
- **order:** Rotationsreihenfolge für die Eulerwinkel als String.

Rückgabewert:

- Die Rotationsmatrix als mehrdimensionales Array.
-

```
def line_wheel_driven_robot(dummy, vel, steps):
```

Simuliert die Bewegung eines radgetriebenen Roboters entlang einer Linie und erzeugt dabei Liniensegmente zur Visualisierung des Pfades.

Bei jedem Schritt wird das Dummy-Objekt gemäß der gegebenen Geschwindigkeit bewegt. In regelmäßigen Abständen (alle 64 Schritte) wird ein Liniensegment vom Startpunkt dieses Abschnitts zum aktuellen Punkt erstellt, um die Trajektorie sichtbar zu machen.

Parameter:

- **dummy:** Ein Mesh-Objekt, das die Position und Orientierung des Roboters repräsentiert.
- **vel:** Ein Geschwindigkeitsvektor $[v_x, v_y, \omega_z]$, bestehend aus Translation in lokaler X/Y-Richtung und Rotation um Z.
- **steps:** Anzahl der Bewegungs-Iterationen (Zeitschritte).

Rückgabewert:

- Eine `three.Group`, die alle erzeugten Liniensegmente enthält (als Trajektorie).
-


```
def move(robot, vel, steps=1):
```

Bewegt ein Roboterobjekt in mehreren Schritten entsprechend der gegebenen Geschwindigkeit und Rotation.

Die Funktion kombiniert Translation und Rotation:

Zuerst wird eine Rotation um die Z-Achse angewendet, basierend auf dem dritten Element des Geschwindigkeitsvektors `vel[2]`. Anschließend wird eine Translation basierend auf der aktuellen Ausrichtung (Z-Rotation) des Roboters ausgeführt. Die Bewegung wird für eine angegebene Anzahl von Schritten (`steps`) wiederholt. Nach jeweils 4 Schritten erfolgt eine kurze Pause zur visuellen Glättung.

Parameter:

- `robot`: Das Objekt (z.B. ein Mesh), das bewegt werden soll. Es muss `quaternion` und `position` Attribute besitzen.
- `vel`: Ein Geschwindigkeitsvektor `[v_x, v_y,  $\omega_z$ ]`, wobei `v_x` und `v_y` die Translation in der lokalen X- und Y-Richtung und `$\omega_z$` die Rotation um die Z-Achse ist (in Radian).
- `steps`: Die Anzahl der Schritte, die die Bewegung ausführen soll (Standard: 1).

Rückgabewert:

- Die finale Position des Roboters nach der Bewegung.
-

```
def order_angles(x, y, z, order):
```

Ordnet die übergebenen Eulerwinkel nach `order` und gibt diese als Array zurück.

Parameter:

- `x`: Eulerwinkel um x.
- `y`: Eulerwinkel um y.
- `z`: Eulerwinkel um z.
- `order`: Die Reihenfolge, in der die Eulerwinkel geordnet werden sollen.

Rückgabewert:

- Die geordneten Eulerwinkel als Array.
-

```
def quaternion_multiply(q1, q2):
```

Multipliziert zwei Quaternions.

Parameter:

- q1: Das erste Quaternion als Liste oder Array $[x, y, z, w]$.
- q2: Das zweite Quaternion als Liste oder Array $[x, y, z, w]$.

Rückgabewert:

- Das Ergebnis der Quaternion-Multiplikation als Liste $[x, y, z, w]$.
-

```
def quaternion_to_euler(x, y, z, w, order='ZYX'):
```

Wandelt ein Quaternion in Eulerwinkel um. Dabei wird die übergebene Rotationsreihenfolge für die Eulerwinkel beachtet.

Parameter:

- x: x-Komponente des Quaternions.
- y: y-Komponente des Quaternions.
- z: z-Komponente des Quaternions.
- w: w-Komponente des Quaternions.
- order: Rotationsreihenfolge für die Eulerwinkel als String.

Rückgabewert:

- Die Eulerwinkel als Array in der Reihenfolge, wie es **order** vorgibt, z.B. bei **order="SZXY"**: $[z, x, y]$.
-

```
def rot_axis_from_rot_mat(rot_mat):
```

Gibt die Rotationsachse von rot_mat zurück.

Parameter:

- rot_mat: Die Rotationsmatrix als mehrdimensionales Array.

Rückgabewert:

- Rotationsachse als normalisierter Vektor z.B. $[x,y,z]$.
-

```
def rot_matrix_to_euler(rot_mat, order='ZYZ'):
```

Wandelt eine Rotationsmatrix in Eulerwinkel um.

Parameter:

- rot_mat: Die Rotationsmatrix.
- order: Rotationsreihenfolge für die Eulerwinkel als String.

Rückgabewert:

- Die Eulerwinkel. Diese werden in der Reihenfolge zurückgegeben, wie es order vorgibt, z. B. order="SZXY" $\rightarrow [Z, X, Y]$.
-

```
def rot_matrix_to_quaternion(rot_mat):
```

Wandelt eine Rotationsmatrix in ein Quaternion um.

Parameter:

- rot_mat: Die Rotationsmatrix als mehrdimensionales Array.

Rückgabewert:

- Das Quaternion als Array.
-

```
def rotate(mesh, angles, order='ZYZ'):
```

Führt eine Rotation eines Mesh-Objekts basierend auf den übergebenen Eulerwinkeln und einer Rotationsreihenfolge durch.

Parameter:

- **mesh:** Das Mesh-Objekt, das rotiert werden soll.
- **angles:** Die Eulerwinkel in Grad, die die Rotation definieren. Die Reihenfolge muss dem angegebenen **order**-Parameter entsprechen.
- **order:** Die Rotationsreihenfolge als String (z.B. "SYZ", "XYZ", etc.). Standardmäßig "SYZ".

Rückgabewert:

- Es wird keine Rückgabe gemacht, da die Rotation direkt auf das **mesh**-Objekt angewendet wird.
-

```
def rotate_animated(mesh, angles, order='ZYZ'):
```

Führt eine animierte lokale Rotation eines Mesh-Objekts durch. Die Rotation erfolgt achsweise entsprechend der angegebenen Reihenfolge.

Beispiel: `order="YXZ"` → `angles=[Winkel um Y, Winkel um X, Winkel um Z]`

Während der Animation wird die Rotation in drei Schritten durchgeführt – einer pro Achse – und am Ende exakt auf das Ziel-Quaternion gesetzt, um numerische Fehler auszugleichen.

Parameter:

- **mesh:** Das 3D-Mesh-Objekt, das rotiert werden soll.
- **angles:** Eine Liste mit Rotationswinkeln (in Grad), die in der Reihenfolge **order** angegeben sind.
- **order:** Die Rotationsreihenfolge (z.B. "SYZ", "YXZ", etc.).

Rückgabewert:

- Keine Rückgabe (void).
-

```
def rotate_global(mesh, angles, order='ZYZ'):
```

Führt eine globale Rotation eines Mesh-Objekts durch, basierend auf den übergebenen Eulerwinkeln und einer Rotationsreihenfolge.

Parameter:

- **mesh:** Das Mesh-Objekt, das rotiert werden soll.
- **angles:** Die Eulerwinkel in Grad, die die Rotation definieren. Die Reihenfolge muss dem angegebenen **order**-Parameter entsprechen.
- **order:** Die Rotationsreihenfolge als String (z.B. "SZYZ", "XYZ", etc.). Standardmäßig "SZYZ".

Rückgabewert:

- Es wird keine Rückgabe gemacht, da die Rotation direkt auf das **mesh**-Objekt angewendet wird.
-

```
def rotate_global_animated(mesh, angles, order='ZYZ'):
```

Führt eine animierte globale Rotation eines Mesh-Objekts durch. Die Drehung erfolgt achsweise gemäß der angegebenen Rotationsreihenfolge (z.B. "SZYZ"). Die Winkel in **angles** müssen in der Reihenfolge der **order**-Zeichen angegeben werden.

Beispiel: Bei **order="YXZ"** → **angles**=[Winkel um Y, Winkel um X, Winkel um Z]

Die Funktion führt die Rotation in drei separaten animierten Phasen durch – jeweils eine für jede Achse in **order**.

Parameter:

- **mesh:** Das 3D-Objekt (Mesh), das rotiert werden soll.
- **angles:** Eine Liste von Rotationswinkeln (in Grad), entsprechend der Reihenfolge in **order**.
- **order:** Die Rotationsreihenfolge als String, z.B. "SZYZ", "YXZ", etc.

Rückgabewert:

- Keine Rückgabe (void).
-

```
def set_rotation(mesh, angles, order='ZYZ'):
```

Setzt die Rotation eines Mesh-Objekts auf die übergebenen Eulerwinkel und die Rotationsreihenfolge.

Parameter:

- **mesh:** Das Mesh-Objekt, dessen Rotation gesetzt werden soll.
- **angles:** Die Eulerwinkel in Grad, die die gewünschte Rotation definieren. Die Reihenfolge muss dem angegebenen **order**-Parameter entsprechen.
- **order:** Die Rotationsreihenfolge als String (z.B. "SZYZ", "XYZ", etc.). Standardmäßig "SZYZ".

Rückgabewert:

- Es wird keine Rückgabe gemacht, da die Rotation direkt auf das **mesh**-Objekt angewendet wird.
-

```
def set_rotation_global(mesh, angles, order='ZYZ'):
```

Setzt die globale Rotation eines Mesh-Objekts auf die übergebenen Eulerwinkel und die Rotationsreihenfolge.

Parameter:

- **mesh:** Das Mesh-Objekt, dessen globale Rotation gesetzt werden soll.
- **angles:** Die Eulerwinkel in Grad, die die gewünschte Rotation definieren. Die Reihenfolge muss dem angegebenen **order**-Parameter entsprechen.
- **order:** Die Rotationsreihenfolge als String (z.B. "SZYZ", "XYZ", etc.). Standardmäßig "SZYZ".

Rückgabewert:

- Es wird keine Rückgabe gemacht, da die Rotation direkt auf das **mesh**-Objekt angewendet wird.
-

```
def set_scale(mesh, scale):
```

Setzt die Skalierung eines Mesh-Objekts.

Parameter:

- **mesh:** Das Mesh-Objekt, dessen Skalierung angepasst werden soll.
- **scale:** Ein Array, das den Skalierungsfaktor für jede Achse (x, y, z) angibt, z.B. [1, 2, 1].

Rückgabewert:

- Keine Rückgabe (void).
-

```
def set_scale_animated(mesh, scale):
```

Setzt die Skalierung eines Mesh-Objekts animiert, indem es schrittweise die Größe verändert.

Parameter:

- **mesh:** Das Mesh-Objekt, dessen Skalierung angepasst werden soll.
- **scale:** Ein Array, das die Ziel-Skalierungswerte für jede Achse (x, y, z) angibt, z.B. [1, 2, 1].

Rückgabewert:

- Keine Rückgabe (void).
-

```
def set_translation(mesh, vec):
```

Setzt die Position eines Mesh-Objekts auf die angegebenen Koordinaten.

Parameter:

- **mesh:** Das Mesh-Objekt, dessen Position gesetzt werden soll.
- **vec:** Der Ziel-Vektor als Array oder Liste [x, y, z], der die neue Position des Meshs im Raum angibt.

Rückgabewert:

- Es wird keine Rückgabe gemacht, da die Position des **mesh**-Objekts direkt gesetzt wird.
-

```
def set_translation_animated(mesh, vec, speed=50.0):
```

Bewegt die Position eines Mesh-Objekts animiert von der aktuellen Position zu einer angegebenen Zielposition.

Parameter:

- **mesh:** Das Mesh-Objekt, dessen Position animiert geändert werden soll.
- **vec:** Der Ziel-Vektor als Array oder Liste `[x, y, z]`, zu dem die Position des Meshs bewegt werden soll.
- **speed:** Die Geschwindigkeit der Animation. Ein höherer Wert bedeutet eine schnellere Bewegung.

Rückgabewert:

- Es wird keine Rückgabe gemacht, da die Position des `mesh`-Objekts animiert geändert wird.
-

```
def slerp_quaternion(q1, q2, t):
```

Führt eine Spherical Linear Interpolation (SLERP) zwischen zwei Quaternionen durch. Interpoliert die Rotation zwischen `q1` und `q2` basierend auf dem Interpolationswert `t`.

Parameter:

- **q1:** Das erste Quaternion, das die Anfangsrotation beschreibt.
- **q2:** Das zweite Quaternion, das die Endrotation beschreibt.
- **t:** Der Interpolationswert, der zwischen 0 und 1 liegen muss. Ein Wert von 0 entspricht der Rotation von `q1` und ein Wert von 1 entspricht der Rotation von `q2`.

Rückgabewert:

- Das interpolierte Quaternion, das die Rotation zwischen `q1` und `q2` bei dem gegebenen Wert von `t` beschreibt.

Fehlerbehandlung:

- **ValueError:** Wenn der Interpolationswert `t` nicht zwischen 0 und 1 liegt.
-


```
def translate(mesh, vec):
```

Verschiebt ein Mesh-Objekt um einen gegebenen Vektor in den drei Raumachsen.

Parameter:

- **mesh:** Das Mesh-Objekt, das verschoben werden soll.
- **vec:** Der Verschiebungsvektor als Array oder Liste `[x, y, z]`, der die Verschiebung in den jeweiligen Raumachsen angibt.

Rückgabewert:

- Es wird keine Rückgabe gemacht, da das **mesh**-Objekt direkt verschoben wird.
-

```
def translate_animated(mesh, vec, speed=50.0):
```

Bewegt die Position eines Mesh-Objekts animiert um einen angegebenen Vektor von der aktuellen Position.

Parameter:

- **mesh:** Das Mesh-Objekt, dessen Position animiert geändert werden soll.
- **vec:** Der Verschiebungs-Vektor als Array oder Liste `[dx, dy, dz]`, um den die Position des Meshs verändert werden soll.
- **speed:** Die Geschwindigkeit der Animation. Ein höherer Wert bedeutet eine schnellere Bewegung.

Rückgabewert:

- Keine Rückgabe.
-

```
class Environment:
```

Eine 3D-Umgebung, die eine Szene mit Kamera, Lichtquellen, Achsen, Gittern und Widgets für die Interaktivität erstellt.

Diese Klasse erstellt eine 3D-Umgebung mit einer Vielzahl von Features, darunter eine Kamera, Lichtquellen, Achsen- und Gitterdarstellung sowie Steuerungen zur Anpassung von Objekten in der Szene (z.B. Rotation, Skalierung, Translation).

Parameter:

- **width:** Die Breite der Ansicht in Pixeln (Standard: 700).
- **height:** Die Höhe der Ansicht in Pixeln (Standard: 500).
- **frame:** Ein 3D-Achsenobjekt, das als Referenzrahmen in der Szene hinzugefügt wird (Standard: `create_axes(8, name="B")`).
- **grid:** Ein Gitterobjekt, das in der Szene angezeigt wird (Standard: `create_grid_XY(14, 0.5)`).
- **up:** Die Richtung der ObenAchse, die die Orientierung der Kamera bestimmt (Standard: `[0, 0, 1]`).

Diese Klasse enthält Methoden, um:

- Die Sichtbarkeit von Gitter und Achsen zu steuern.
- Interaktive Widgets für Objekte zu erstellen (Translation, Rotation, Skalierung).
- Objekte der Szene hinzuzufügen.
- Globale oder lokale Transformationen auf Objekte anzuwenden.

Weitere Features:

- Die Umgebung kann mit einer interaktiven Steuerung für Kamera und Objekte angezeigt werden.
- Widgets für die Manipulation von Objekten (Translation, Rotation, Skalierung) können zur Szene hinzugefügt werden.

```
def add(self, objekts):
```

Fügt ein oder mehrere Objekte zur Szene hinzu.

Parameter:

- **objekts:** Ein einzelnes Objekt oder eine Liste von Objekten, die zur Szene hinzugefügt werden.
-

```
def add_gizmo_controls (self, obj, translation=True,  
rotation=True, scale=True):
```

Fügt ein Gizmo-Steurelement zur Manipulation eines Objekts in der Umgebung hinzu (Translation, Rotation, Skalierung).

Parameter:

- **obj**: Das Objekt, das manipuliert werden soll.
 - **translation**: Wenn True, werden Schieberegler für die Translation angezeigt.
 - **rotation**: Wenn True, werden Schieberegler für die Rotation angezeigt.
 - **scale**: Wenn True, werden Schieberegler für die Skalierung angezeigt.
-

```
def add_widget(self, widget):
```

Fügt ein Widget zur Umgebung hinzu. Dabei kann es sich auch um ein Bündel von Widgets in einer HBox oder einer VBox handeln.

Parameter:

- **widget**: Das Widget, das der Umgebung hinzugefügt werden soll.
-

```
def set_frame_widgets(self, bool):
```

Aktiviert oder deaktiviert die Anzeige von Frame-Widgets.

Parameter:

- **bool**: Wenn True, werden die Widgets angezeigt, andernfalls ausgeblendet.
-

```
def toggle_axes(self, change):
```

Schaltet die Sichtbarkeit der Achsen um.

Parameter:

- `change`: Das Ereignis, das diese Funktion auslöst (wird nicht genutzt).
-

```
def toggle_grid(self, change):
```

Schaltet die Sichtbarkeit des Gitters um.

Parameter:

- `change`: Das Ereignis, das diese Funktion auslöst (wird nicht genutzt).
-